JavaScript Lab: Solved Reference

CS 436 Lab Sessions 1-10, Sections B and C, and the JS Lab Exercise Sheet (Sets A-D). 197 questions, all code run and tested.

- 0. How to use this in the exam
- Question Index
- 1. JavaScript Basics: output, variables, data types, operators
- 2. Conditions and Loops
- 3. Number and Math Programs
- 4. Functions and Recursion
- 5. Arrays: create, access, add, remove
- 6. Arrays: search, sort, duplicates, set operations
- 7. Arrays: transform and algorithms
- 8. 2D Arrays (Matrices)
- 9. Strings
- 10. Regular Expressions and Validation
- 11. Objects
- 12. Classes, Inheritance, OOP, Frameworks
- 13. Built-in Objects: Boolean, Symbol, Error, Date, Map, Set, Cookies
- 14. BOM: window, navigator, location, history, screen
- 15. DOM and Events
- 16. Forms and Projects (Section B)

0. How to use this in the exam

1. **Find the closest solved question.** Look up the question number in the **Question Index** (lab session, Set A–D, Section B/C), or browse by topic in the contents.
2. **Copy the pattern, change the input.** Every program has its input at the top (`let n = 29;`), so you only change the values or the condition.
3. **Need input or output from the user?** Swap in one of the templates below.

Input templates

```
let n = Number(prompt("Enter a number"));          // number from popup
let s = prompt("Enter text");                      // string from popup
let v = Number(document.getElementById("num").value); // number from an HTML input box
let arr = "10,20,30".split(",").map(Number);       // comma list -> number array
```

Output templates

```
console.log(result);                               // console
document.write(result + "<br>");                   // on the page while loading
document.getElementById("out").innerHTML = result;  // into an element (after a click)
alert(result);                                     // popup
```

Form + button skeleton (wrap any program in this)

```
<input type="number" id="num">
<button type="button" onclick="run()">Run</button>
<p id="out"></p>
<script>
function run() {
  let n = Number(document.getElementById("num").value);
  // ... logic from any program in this PDF ...
  document.getElementById("out").innerHTML = "Result: " + n;
}
</script>
```

Five rules that prevent most mistakes

- `.value` and `prompt()` always give a **string**, so wrap them in `Number()` before doing maths.
- To sort numbers, always use `sort((a, b) => a - b)`. Plain `sort()` sorts them as text.
- Check `if (isNaN(n))` before calculating.
- Use `===`, not `==`.
- A button inside a `<form>` submits the form and reloads the page. Use `type="button"`, or `return false`.

Question Index

Find your question here; the **bold number** is the section in this PDF (click it in the digital copy).

CS 436 Lab Tutorial, Section A (Sessions 1-10)

Q	Question	Solved in
S1.1	Basic programs: constructs, functions, closures	**1.1**, **4.6**
S1.2	Welcome message using alert()	**1.2**
S1.3	Literals, identifiers, variable declarations	**1.3**
S1.4	Declare and print all primitive data types	**1.4**
S2.1	Insert dashes between even digits (025468)	**2.5**
S2.2	var, let, const scopes	**1.5**
S2.3	Calculator + - * / % **	**1.7**
S2.4	typeof to check data types	**1.6**
S2.5	Even/odd with if-else	**2.1**
S2.6	Grading system with switch-case	**2.2**
S2.7	Primes from 1 to 100 with for loop	**3.2**
S3.1	Create and access object properties	**11.1**
S3.2	Accessors and mutators	**11.3**
S3.3	Iterate object properties with for-in	**11.4**
S3.4	Static methods and nullify object	**11.5**
S4.1	UI refresh with a modern stack (React/Vue)	**12.4**
S4.2	Data persistence with JavaScript	**12.5**
S4.3	Person class with method	**12.1**
S4.4	Extend to Student class with marks	**12.2**
S5.1	Last element / last n elements of an array	**5.3**
S5.2	Recursive factorial	**4.2**
S5.3	Sum array elements with a loop	**5.14**
S5.4	Array by literal and constructor	**5.1**
S5.5	Array after r right rotations	**7.9**
S6.1	Most frequent item in an array	**6.11**
S6.2	Add items to a blank array and display	**5.6**
S6.3	Remove duplicates ignoring case	**6.7**
S6.4	map(), filter(), reduce()	**7.1**
S6.5	Sort student marks descending	**6.2**
S6.6	Create and access 2D array (matrix)	**8.1**, **8.3**, **8.4**
S7.1	Object-oriented properties of JS	**12.3**
S7.2	Browser info using navigator	**14.1**
S7.3	location.href and history.back()	**14.2**
S7.4	Screen resolution using screen	**14.3**
S8.1	Manipulate the BOM and the DOM	**14.5**, **15.2**
S8.2	Build DOM tree, getElementById()	**15.1**
S8.3	Add row to table dynamically	**15.3**
S8.4	Change text on button click (addEventListener)	**15.4**
S9.1	5 random numbers with Math	**3.7**
S9.2	Validate form fields with JS	**10.3**, **10.6**
S9.3	Boolean object: truthy/falsy	**13.1**
S9.4	Symbol as hidden property (not in for-in)	**13.2**
S9.5	Custom error with try…catch + stack trace	**13.3**
S9.6	Set, retrieve, delete cookies	**13.10**
S10.1	6-digit OTP with Number and Math	**3.8**

S10.2	Date as YYYY-MM-DD	**13.5**
S10.3	Map of student marks	**13.6**
S10.4	Email validation with RegExp test()	**10.2**
S10.5	Set avoids duplicates	**13.7**

CS 436 Lab Tutorial, Section B (Projects) and Section C

Q	Question	Solved in
PB1	Form Validation Page	**10.6**, **16.18**
PB2	Speech Detection / Text-to-speech	**16.12**
PB3	BMI Calculator	**16.3**
PB4	Digital Clock	**16.4**
PB5	Single Page App	**16.13**
PB6	Clock (alarm, stopwatch, timer)	**16.5**
PB7	Scramble word game	**16.10**
PB8	Calculator	**16.6**
PB9	Notes / To-Do List App	**16.7**
PB10	Budget Application	**16.11**
PB11	E-Commerce	**16.14**
PB12	Quiz App	**16.8**
PB13	Movie App	**16.15**
PB14	File Sharing App (encryption)	**16.16**
PB15	Chat Application	**16.17**
PB16	Quotes Generator	**16.9**
PB17	To-Do List	**16.7**
SC1a	USN validation with event handler and alerts	**10.7**
SC1b	USN + semester (1 to 8)	**10.7**
SC2a	Stacked paragraphs rise to top on hover	**15.7**
SC2b	Paragraph returns to original position	**15.7**

JS Lab Exercise Sheet, Set A: Basics (1-10)

Q	Question	Solved in
A1	Generate Random Number in Given Range	**3.6**
A2	Check Prime Number	**3.1**
A3	Find Largest of Three Numbers	**2.3**
A4	Print All Prime Numbers in an Interval	**3.2**
A5	Check if Two Numbers have Same Last Digit	**2.4**
A6	Guess Number	**2.7**
A7	Split a Number into Individual Digits	**3.4**
A8	Find the Missing Number	**3.5**
A9	Find the Factors of a Number	**3.3**
A10	Print All Prime Numbers in an Interval	**3.2**

JS Lab Exercise Sheet, Set B: Math and Recursion (1-22)

Q	Question	Solved in
B1	Compute Power of a Number	**3.9**
B2	Find the Square Root	**3.10**
B3	Solve Quadratic Equations	**3.11**
B4	Convert Km/hr to miles/hr and vice versa	**3.12**
B5	Factorial of a Number	**4.1**
B6	Print Multiplication Table of a Number	**2.6**
B7	Print Fibonacci Series	**4.3**

B8	Find GCD or HCF of Two Numbers	**3.13**
B9	Find LCM of Two Numbers	**3.14**
B10	Find the Factors of a Number	**3.3**
B11	Shuffle Deck of Cards	**7.12**
B12	Display Fibonacci Sequence Using Recursion	**4.4**
B13	Find Factorial of a Number using Recursion	**4.2**
B14	Convert Decimal to Binary	**3.15**
B15	Find Sum of Natural Numbers using Recursion	**4.5**
B16	Convert a Float Number to Whole Number	**3.16**
B17	Find the Quotient and Remainder by Dividing an Integer	**3.17**
B18	Convert a Negative Number to Positive	**3.18**
B19	Round off a Number to Next Multiple of 5	**3.19**
B20	Find Factorial of a Number	**4.1**
B21	Perform Simple Mathematical Calculation	**1.7**
B22	Calculate nPr (Permutations)	**3.20**

JS Lab Exercise Sheet, Set C: Arrays (1-49)

Q	Question	Solved in
C1	Get the First and Last Item in an Array	**5.2**
C2	Add New Elements at the Beginning of an Array	**5.5**
C3	Remove Elements from an Array	**5.7**
C4	Reverse an Array	**5.10**
C5	Empty an Array	**5.11**
C6	Check Whether an Array Includes a Particular Value or Not	**5.12**
C7	Insert an Item into an Array at Specific Index	**5.8**
C8	Convert a Number into Array	**3.4**
C9	Find the Sum of All Elements of Given Array	**5.14**
C10	Sort Words in Alphabetical Order	**6.3**
C11	Remove Duplicate Elements from an Array	**6.6**
C12	Find Common Items from Two Arrays	**6.13**
C13	Replace an Item from an Array	**5.9**
C14	Check Object is an Array or Not	**5.15**
C15	Generate a Range of Numbers and Characters	**7.4**
C16	Find Duplicate Elements in an Array	**6.8**
C17	Sort Numeric Array	**6.1**
C18	Remove Duplicate Elements from an Array	**6.6**
C19	Get All Unique Values (Remove Duplicates) in an Array	**6.6**
C20	Add Elements to an Existing Array Dynamically	**5.6**
C21	Merge Two Arrays and Remove Duplicate Items	**6.14**
C22	Sort Array of Objects by String Property Value	**6.4**
C23	Create Two Dimensional Array	**8.1**, **8.2**
C24	Splice an Array Without Mutating the Original Array	**7.5**
C25	Extract Value of a Property as Array from Array of Objects	**7.3**

C26	Compare Two Arrays	**6.15**
C27	Select a Random Element from Array	**5.16**
C28	Create an Array using Intersection of two Arrays	**6.13**
C29	Split an Array into Chunks	**7.6**
C30	Convert Comma Separated String to Array	**5.18**
C31	Copy Array Items into Another Array	**5.17**
C32	Find the Array Index with a Value	**5.13**
C33	Merge Two Arrays and Remove Duplicate Items	**6.14**
C34	Count Occurrences of All Items in an Array	**6.10**
C35	Filter the Non-unique Values in an Array	**6.9**
C36	Get N Largest Elements from an Array	**6.16**
C37	Clone an Array	**5.17**
C38	find Smallest Difference Triplet from Three Arrays	**7.13**
C39	Find Shortest Distance between Two Words in an Array of Words	**7.14**
C40	Find the Most Frequently Occurring Element in an Array	**6.11**
C41	Find Maximum Profit by Buying and Selling a Share at Most Twice using Array	**7.15**
C42	Calculate the Cumulative Sum of Elements in an Array	**7.7**
C43	Merge Two Arrays Without Creating a New Array	**7.8**
C44	K-th Largest Sum Contiguous Sub-array	**7.16**
C45	Construct an Array from its pair-sum Array	**7.17**
C46	Check if an Array Contains only Unique Values	**6.12**
C47	Find the Most Frequent Element in an Array	**6.11**
C48	Left Rotate by One in an Array	**7.10**
C49	Find the Rotation Count in Rotated Sorted array in JavaScript	**7.11**

JS Lab Exercise Sheet, Set D: Strings (1-46)

Q	Question	Solved in
D1	Count Words of a String	**9.2**
D2	Check if a String Contains any Whitespace Characters	**9.3**
D3	Reverse a String	**9.4**
D4	Replace All Occurrences of a String	**9.5**
D5	Check if a String Starts With Another String	**9.6**
D6	Create Multi Line Strings	**9.7**
D7	Format Numbers as Currency String	**9.8**
D8	Trim a String at Beginning or Ending	**9.9**
D9	Delete First Character of a String	**9.10**
D10	Get the Last Character of a String	**9.10**
D11	Convert an Object to String	**9.11**
D12	Check Whether a String Contains a Substring	**9.6**
D13	Compare Strings	**9.12**
D14	Encode/Decode String to Base64	**9.13**
D15	Replace Characters of a String	**9.5**
D16	Check Whether a Passed String is Palindrome or Not	**9.14**

D17	Replace Characters of a String	**9.5**
D18	Get the Number of Occurrences of Each Letter in Specified String	**9.15**
D19	Generate a Random String	**9.16**
D20	Make First Letter of a String Uppercase	**9.17**
D21	Get the Number of Vowels in a String	**9.18**
D22	Remove Spaces from a String	**9.19**
D23	Count String Occurrence in String	**9.20**
D24	Compare the Case Insensitive Strings	**9.12**
D25	Generate All Combinations of a String	**9.21**
D26	Find Unique Characters of a String	**9.22**
D27	Convert User Input String into Regular Expression	**10.4**
D28	Convert Comma Separated String to Array	**5.18**
D29	Strip All Non-numeric Characters from String	**9.23**
D30	Convert String to Title Case	**9.17**
D31	Remove All Non-ASCII Characters from the String	**9.24**
D32	Convert String into Float	**9.23**
D33	Check Whether a String Contains a Substring	**9.6**
D34	Split Multiline String into an Array of Lines	**9.25**
D35	Find the Longest Word Within the String	**9.26**
D36	Get Nth Occurrence of a String	**9.27**
D37	Replace All Dots in a String	**9.5**
D38	Check a Given String is an Anagram of Another String	**9.28**
D39	Check if String Follows Order of Characters Defined by a Pattern or Not	**9.30**
D40	Extract Email Addresses from a String	**10.5**
D41	Check if a String Contains Any Digit Characters	**9.31**
D42	Count the Occurrences of Each Character	**9.15**
D43	Find i’th Index Character in a Binary String Obtained After n Iterations	**9.32**
D44	find Smallest and Largest Word in a String	**9.26**
D45	Find Minimum Number of Manipulations to Make Two Strings Anagram Without Deletion of Character	**9.29**
D46	Get All Substrings of the Given String	**9.33**

1. JavaScript Basics: output, variables, data types, operators

1.1 Basic constructs: function, loop, closure

Asked as: Lab S1.1

```
// function
function add(a, b) {
  return a + b;
}
// loop + condition
for (let i = 1; i <= 5; i++) {
  if (i % 2 === 0) console.log(i + " is even");
}
// closure: inner function remembers count
function counter() {
  let count = 0;
  return function () {
    count++;
    return count;
  };
}
let c = counter();
console.log("Sum =", add(4, 6));
console.log(c(), c(), c());
```

Output:

```
2 is even
4 is even
Sum = 10
1 2 3
```

1.2 Welcome message with alert()

Asked as: Lab S1.2

```
<!DOCTYPE html>
<html>
<body>
  <h2>My Page</h2>
  <script>
    alert("Welcome to Front End Design Lab!");
  </script>
</body>
</html>
```

Output: A popup that says *Welcome to Front End Design Lab!*

1.3 Literals, identifiers, variable declarations

Asked as: Lab S1.3

```
var name = "Amit";              // string literal, identifier: name
let age = 21;                   // number literal
const PI = 3.14;                // constant
let isStudent = true;           // boolean literal
let marks = [80, 90, 75];       // array literal
let student = { roll: 101 };    // object literal
console.log(name, age, PI, isStudent);
console.log(marks, student);
```

Output:

```
Amit 21 3.14 true
[80, 90, 75] { roll: 101 }
```

1.4 Declare and print all primitive data types

Asked as: Lab S1.4

```
let num = 42;
let str = "Hello";
let bool = true;
let undef;
let empty = null;
let big = 12345678901234567890n;
let sym = Symbol("id");
console.log(num, typeof num);
console.log(str, typeof str);
console.log(bool, typeof bool);
console.log(undef, typeof undef);
console.log(empty, typeof empty);
console.log(big, typeof big);
console.log(sym.toString(), typeof sym);
```

Output:

```
42 number
Hello string
true boolean
undefined undefined
null object
12345678901234567890n bigint
Symbol(id) symbol
```

1.5 var, let, const scope

Asked as: Lab S2.2

```
function scopeTest() {
  if (true) {
    var a = 10;      // function scoped
    let b = 20;      // block scoped
    const c = 30;    // block scoped
    console.log("Inside block:", a, b, c);
  }
  console.log("Outside block, var a =", a);
  try {
    console.log(b);
  } catch (e) {
    console.log("Outside block, b is not accessible:", e.name);
  }
}
scopeTest();
```

Output:

```
Inside block: 10 20 30
Outside block, var a = 10
Outside block, b is not accessible: ReferenceError
```

1.6 typeof to check data types

Asked as: Lab S2.4

```
console.log(typeof 25);
console.log(typeof "JS");
console.log(typeof false);
console.log(typeof undefined);
console.log(typeof null);          // "object" (known JS bug)
console.log(typeof [1, 2]);        // arrays are objects
console.log(typeof { a: 1 });
console.log(typeof function () {});
console.log(Array.isArray([1, 2])); // correct way to check for an array
```

Output:

```
number
string
boolean
undefined
object
object
object
function
true
```

1.7 Calculator for + - * / % **

Asked as: Lab S2.3 · Sheet B21

```
function calculate(a, op, b) {
  switch (op) {
    case "+":  return a + b;
    case "-":  return a - b;
    case "*":  return a * b;
    case "/":  return b === 0 ? "Cannot divide by zero" : a / b;
    case "%":  return a % b;
    case "**": return a ** b;
    default:   return "Invalid operator";
  }
}
// input: let a = Number(prompt("a")), op = prompt("op"), b = Number(prompt("b"));
let ops = ["+", "-", "*", "/", "%", "**"];
for (let op of ops) {
  console.log("15 " + op + " 4 = " + calculate(15, op, 4));
}
```

Output:

```
15 + 4 = 19
15 - 4 = 11
15 * 4 = 60
15 / 4 = 3.75
15 % 4 = 3
15 ** 4 = 50625
```

2. Conditions and Loops

2.1 Even or odd (if-else)

Asked as: Lab S2.5

```
let n = 7;   // or: Number(prompt("Enter a number"))
if (n % 2 === 0) {
  console.log(n + " is Even");
} else {
  console.log(n + " is Odd");
}
```

Output:

```
7 is Odd
```

2.2 Grading system with switch-case

Asked as: Lab S2.6

```
function getGrade(marks) {
  let grade;
  switch (true) {          // switch(true) lets each case be a condition
    case marks >= 90: grade = "A"; break;
    case marks >= 75: grade = "B"; break;
    case marks >= 60: grade = "C"; break;
    case marks >= 40: grade = "D"; break;
    default:          grade = "F";
  }
  return grade;
}
console.log(95, getGrade(95));
console.log(82, getGrade(82));
console.log(35, getGrade(35));
```

Output:

```
95 A
82 B
35 F
```

2.3 Largest of three numbers

Asked as: Sheet A3

```
function largest(a, b, c) {
  if (a >= b && a >= c) return a;
  else if (b >= a && b >= c) return b;
  else return c;
}
console.log(largest(12, 45, 30));
console.log(Math.max(12, 45, 30));   // built-in way
```

Output:

```
45
45
```

2.4 Two numbers have the same last digit?

Asked as: Sheet A5

```
function sameLastDigit(a, b) {
  return Math.abs(a) % 10 === Math.abs(b) % 10;   // abs handles negatives
}
console.log(sameLastDigit(27, 57));
console.log(sameLastDigit(27, 58));
```

Output:

```
true
false
```

2.5 Insert dashes between consecutive even digits

Asked as: Lab S2.1

```
// read as a STRING so the leading 0 is kept (Number("025468") loses it)
let num = "025468";    // or: prompt("Enter a number")
let result = num[0];
for (let i = 1; i < num.length; i++) {
  if (num[i] % 2 === 0 && num[i - 1] % 2 === 0) {
    result += "-";
  }
  result += num[i];
}
console.log(result);
```

Output:

```
0-254-6-8
```

2.6 Multiplication table

Asked as: Sheet B6

```
let n = 7;
for (let i = 1; i <= 10; i++) {
  console.log(n + " x " + i + " = " + n * i);
}
```

Output:

```
7 x 1 = 7
7 x 2 = 14
7 x 3 = 21
7 x 4 = 28
7 x 5 = 35
7 x 6 = 42
7 x 7 = 49
7 x 8 = 56
7 x 9 = 63
7 x 10 = 70
```

2.7 Guess the number game

Asked as: Sheet A6

```
// Browser version: uses prompt and alert in a loop
let secret = Math.floor(Math.random() * 100) + 1;   // 1 to 100
let attempts = 0;
while (true) {
  let guess = Number(prompt("Guess a number between 1 and 100:"));
  attempts++;
  if (isNaN(guess)) {
    alert("Please enter a number");
  } else if (guess > secret) {
    alert("Too high!");
  } else if (guess < secret) {
    alert("Too low!");
  } else {
    alert("Correct! You took " + attempts + " attempts.");
    break;
  }
}
```

Output: popups say *Too high!* or *Too low!* until you guess right, then *Correct! You took N attempts.*

3. Number and Math Programs

3.1 Check prime number

Asked as: Sheet A2

```
function isPrime(n) {
  if (n <= 1) return false;
  for (let i = 2; i * i <= n; i++) {   // only need to check up to sqrt(n)
    if (n % i === 0) return false;
  }
  return true;
}
console.log(29, isPrime(29));
console.log(30, isPrime(30));
```

Output:

```
29 true
30 false
```

3.2 All primes in an interval (and 1 to 100)

Asked as: Sheet A4 · Sheet A10 · Lab S2.7

```
function primesInRange(start, end) {
  let result = [];
  for (let n = start; n <= end; n++) {
    let isPrime = n > 1;
    for (let i = 2; i * i <= n; i++) {
      if (n % i === 0) { isPrime = false; break; }
    }
    if (isPrime) result.push(n);
  }
  return result;
}
console.log(primesInRange(10, 40).join(" "));
console.log(primesInRange(1, 100).join(" "));
```

Output:

```
11 13 17 19 23 29 31 37
2 3 5 7 11 13 17 19 23 29 31 37 41 43 47 53 59 61 67 71 73 79 83 89 97
```

3.3 Factors of a number

Asked as: Sheet A9 · Sheet B10

```
function factors(n) {
  let result = [];
  for (let i = 1; i <= n; i++) {
    if (n % i === 0) result.push(i);
  }
  return result;
}
console.log(factors(36));
```

Output:

```
[1, 2, 3, 4, 6, 9, 12, 18, 36]
```

3.4 Split a number into digits

Asked as: Sheet A7 · Sheet C8

```
// Method 1: arithmetic
function splitDigits(n) {
  n = Math.abs(n);
  if (n === 0) return [0];
  let digits = [];
  while (n > 0) {
    digits.unshift(n % 10);        // add the last digit at the front
    n = Math.floor(n / 10);
  }
  return digits;
}
console.log(splitDigits(4508));
// Method 2: string
console.log(String(4508).split("").map(Number));
```

Output:

```
[4, 5, 0, 8]
[4, 5, 0, 8]
```

3.5 Find the missing number (1 to n, one missing)

Asked as: Sheet A8

```
function findMissing(arr, n) {
  let expected = (n * (n + 1)) / 2;   // sum of 1..n
  let actual = 0;
  for (let x of arr) actual += x;
  return expected - actual;
}
console.log(findMissing([1, 2, 4, 5, 6], 6));
```

Output:

```
3
```

3.6 Random number in a given range

Asked as: Sheet A1

```
function randomInRange(min, max) {
  // Math.random() gives 0 <= r < 1
  return Math.floor(Math.random() * (max - min + 1)) + min;
}
console.log(randomInRange(10, 50));
```

Output (random, so yours will differ):

```
44
```

3.7 Generate 5 random numbers with Math

Asked as: Lab S9.1

```
let nums = [];
for (let i = 0; i < 5; i++) {
  nums.push(Math.floor(Math.random() * 100) + 1);   // 1 to 100
}
console.log(nums);
```

Output (random, so yours will differ):

```
[82, 89, 41, 58, 24]
```

3.8 Random 6-digit OTP (Number + Math, toFixed, parseInt)

Asked as: Lab S10.1

```
let raw = 100000 + Math.random() * 900000;   // 100000.xx to 999999.xx
let otp = parseInt(raw.toFixed(0));          // toFixed(0) rounds to a string, parseInt makes it a number
if (otp > 999999) otp = 999999;              // rounding edge case
console.log("OTP:", otp, "| digits:", String(otp).length);
```

Output (random, so yours will differ):

```
OTP: 413386 | digits: 6
```

3.9 Power of a number

Asked as: Sheet B1

```
function power(base, exp) {           // loop
  let result = 1;
  for (let i = 1; i <= exp; i++) result *= base;
  return result;
}
function powerRec(base, exp) {        // recursion
  if (exp === 0) return 1;
  return base * powerRec(base, exp - 1);
}
console.log(power(2, 10), powerRec(2, 10), Math.pow(2, 10), 2 ** 10);
```

Output:

```
1024 1024 1024 1024
```

3.10 Square root

Asked as: Sheet B2

```
console.log(Math.sqrt(144));
// without Math.sqrt: Newton's method
function mySqrt(n) {
  if (n < 0) return NaN;
  let x = n;
  while (Math.abs(x * x - n) > 0.000001) {
    x = (x + n / x) / 2;
  }
  return x;
}
console.log(mySqrt(144).toFixed(4), mySqrt(2).toFixed(4));
```

Output:

```
12
12.0000 1.4142
```

3.11 Solve a quadratic equation

Asked as: Sheet B3

```
function quadratic(a, b, c) {
  if (a === 0) return "Not a quadratic equation";
  let d = b * b - 4 * a * c;            // discriminant
  if (d > 0) {
    let r1 = (-b + Math.sqrt(d)) / (2 * a);
    let r2 = (-b - Math.sqrt(d)) / (2 * a);
    return "Real and distinct: " + r1 + ", " + r2;
  } else if (d === 0) {
    return "Real and equal: " + (-b / (2 * a));
  } else {
    let re = (-b / (2 * a)).toFixed(2);
    let im = (Math.sqrt(-d) / (2 * a)).toFixed(2);
    return "Complex: " + re + " + " + im + "i, " + re + " - " + im + "i";
  }
}
console.log(quadratic(1, -5, 6));
console.log(quadratic(1, 2, 1));
console.log(quadratic(1, 2, 5));
```

Output:

```
Real and distinct: 3, 2
Real and equal: -1
Complex: -1.00 + 2.00i, -1.00 - 2.00i
```

3.12 km/h to miles/h and back

Asked as: Sheet B4

```
function kmphToMph(k) { return k * 0.621371; }
function mphToKmph(m) { return m / 0.621371; }
console.log("100 km/h =", kmphToMph(100).toFixed(2), "mph");
console.log("60 mph =", mphToKmph(60).toFixed(2), "km/h");
```

Output:

```
100 km/h = 62.14 mph
60 mph = 96.56 km/h
```

3.13 GCD / HCF of two numbers

Asked as: Sheet B8

```
function gcd(a, b) {          // Euclid's algorithm
  while (b !== 0) {
    let temp = b;
    b = a % b;
    a = temp;
  }
  return a;
}
console.log(gcd(36, 60));
```

Output:

```
12
```

3.14 LCM of two numbers

Asked as: Sheet B9

```
function gcd(a, b) { return b === 0 ? a : gcd(b, a % b); }
function lcm(a, b) { return (a * b) / gcd(a, b); }
console.log(lcm(12, 18));
```

Output:

```
36
```

3.15 Decimal to binary

Asked as: Sheet B14

```
console.log((25).toString(2));          // built-in
function decToBin(n) {                  // manual: divide by 2, collect remainders
  if (n === 0) return "0";
  let bin = "";
  while (n > 0) {
    bin = (n % 2) + bin;
    n = Math.floor(n / 2);
  }
  return bin;
}
console.log(decToBin(25));
console.log(parseInt("11001", 2));      // binary back to decimal
```

Output:

```
11001
11001
25
```

3.16 Float to whole number

Asked as: Sheet B16

```
let x = 7.68;
console.log("round:", Math.round(x));
console.log("floor:", Math.floor(x));
console.log("ceil :", Math.ceil(x));
console.log("trunc:", Math.trunc(x));
console.log("parseInt:", parseInt(x));
console.log("toFixed(0):", x.toFixed(0), "(string)");
```

Output:

```
round: 8
floor: 7
ceil : 8
trunc: 7
parseInt: 7
toFixed(0): 8 (string)
```

3.17 Quotient and remainder

Asked as: Sheet B17

```
let dividend = 47, divisor = 5;
let quotient = Math.floor(dividend / divisor);
let remainder = dividend % divisor;
console.log("Quotient =", quotient, ", Remainder =", remainder);
```

Output:

```
Quotient = 9 , Remainder = 2
```

3.18 Negative number to positive

Asked as: Sheet B18

```
let n = -45;
console.log(Math.abs(n));
console.log(n < 0 ? -n : n);
```

Output:

```
45
45
```

3.19 Round to next multiple of 5

Asked as: Sheet B19

```
function roundToNext5(n) {
  return Math.ceil(n / 5) * 5;
}
console.log(roundToNext5(12), roundToNext5(20), roundToNext5(23));
```

Output:

```
15 20 25
```

3.20 nPr (and nCr)

Asked as: Sheet B22

```
function fact(n) {
  let f = 1;
  for (let i = 2; i <= n; i++) f *= i;
  return f;
}
function nPr(n, r) { return fact(n) / fact(n - r); }
function nCr(n, r) { return fact(n) / (fact(r) * fact(n - r)); }
console.log("5P2 =", nPr(5, 2), "| 5C2 =", nCr(5, 2));
```

Output:

```
5P2 = 20 | 5C2 = 10
```

4. Functions and Recursion

4.1 Factorial (loop)

Asked as: Sheet B5 · Sheet B20

```
function factorial(n) {
  if (n < 0) return "Not defined for negative numbers";
  let f = 1;
  for (let i = 2; i <= n; i++) f *= i;
  return f;
}
console.log("6! =", factorial(6));
```

Output:

```
6! = 720
```

4.2 Factorial (recursion)

Asked as: Lab S5.2 · Sheet B13

```
function factRec(n) {
  if (n <= 1) return 1;            // base case
  return n * factRec(n - 1);       // recursive call
}
console.log("5! =", factRec(5));
```

Output:

```
5! = 120
```

4.3 Fibonacci series (loop)

Asked as: Sheet B7

```
function fibonacci(n) {
  let a = 0, b = 1, series = [];
  for (let i = 0; i < n; i++) {
    series.push(a);
    let next = a + b;
    a = b;
    b = next;
  }
  return series;
}
console.log(fibonacci(10).join(" "));
```

Output:

```
0 1 1 2 3 5 8 13 21 34
```

4.4 Fibonacci (recursion)

Asked as: Sheet B12

```
function fib(n) {
  if (n <= 1) return n;
  return fib(n - 1) + fib(n - 2);
}
let out = "";
for (let i = 0; i < 10; i++) out += fib(i) + " ";
console.log(out);
```

Output:

```
0 1 1 2 3 5 8 13 21 34
```

4.5 Sum of natural numbers (recursion)

Asked as: Sheet B15

```
function sumNatural(n) {
  if (n <= 0) return 0;
  return n + sumNatural(n - 1);
}
console.log("Sum 1..10 =", sumNatural(10));
```

Output:

```
Sum 1..10 = 55
```

4.6 Function types: declaration, expression, arrow, default, rest, IIFE

Asked as: Lab S1.1

```
function greet(name = "Guest") { return "Hi " + name; }      // declaration + default
const square = function (x) { return x * x; };               // expression
const cube = x => x * x * x;                                 // arrow
function total(...nums) {                                    // rest parameter
  return nums.reduce((s, x) => s + x, 0);
}
(function () { console.log("IIFE runs immediately"); })();   // IIFE
console.log(greet(), greet("Riya"), square(4), cube(3), total(1, 2, 3, 4));
```

Output:

```
IIFE runs immediately
Hi Guest Hi Riya 16 27 10
```

5. Arrays: create, access, add, remove

5.1 Declare an array: literal and constructor

Asked as: Lab S5.4

```
let fruits = ["Apple", "Mango", "Banana"];   // literal (preferred)
let nums = new Array(1, 2, 3, 4);            // constructor
let empty5 = new Array(5);                   // TRAP: 5 empty slots, not [5]
console.log(fruits, fruits.length);
console.log(nums);
console.log(empty5.length, Array.of(5));
```

Output:

```
['Apple', 'Mango', 'Banana'] 3
[1, 2, 3, 4]
5 [5]
```

5.2 First and last item

Asked as: Sheet C1

```
let arr = [10, 20, 30, 40, 50];
console.log("First:", arr[0]);
console.log("Last :", arr[arr.length - 1]);
console.log("Last with at():", arr.at(-1));
```

Output:

```
First: 10
Last : 50
Last with at(): 50
```

5.3 Last element, or last n elements

Asked as: Lab S5.1

```
function last(arr, n) {
  if (n === undefined) return arr[arr.length - 1];
  if (n <= 0) return [];
  return arr.slice(-n);
}
console.log(last([7, 9, 0, -2]));
console.log(last([7, 9, 0, -2], 3));
```

Output:

```
-2
[9, 0, -2]
```

5.4 First element, or first n elements

Asked as: Theory M3-Q19

```
function first(arr, n) {
  if (n === undefined) return arr[0];
  if (n <= 0) return [];
  return arr.slice(0, n);
}
console.log(first([7, 9, 0, -2]));
console.log(first([7, 9, 0, -2], 3));
```

Output:

```
7
[7, 9, 0]
```

5.5 Add elements at the beginning

Asked as: Sheet C2

```
let arr = [30, 40];
arr.unshift(10, 20);          // changes arr
console.log(arr);
let newArr = [5, ...arr];     // without changing arr
console.log(newArr);
```

Output:

```
[10, 20, 30, 40]
[5, 10, 20, 30, 40]
```

5.6 Add items to a blank array dynamically and display

Asked as: Lab S6.2 · Sheet C20

```
let list = [];
list.push("HTML");
list.push("CSS", "JavaScript");
list[list.length] = "React";     // index assignment also adds at the end
for (let i = 0; i < list.length; i++) {
  console.log((i + 1) + ". " + list[i]);
}
```

Output:

```
1. HTML
2. CSS
3. JavaScript
4. React
```

5.7 Remove elements

Asked as: Sheet C3

```
let arr = [10, 20, 30, 40, 50];
arr.pop();              // remove last
arr.shift();            // remove first
console.log(arr);
arr.splice(1, 1);       // remove 1 element at index 1
console.log(arr);
let a2 = [10, 20, 30];
let i = a2.indexOf(20); // remove by value
if (i !== -1) a2.splice(i, 1);
console.log(a2);
console.log([1, 2, 3, 4].filter(x => x !== 3));   // remove by condition (new array)
```

Output:

```
[20, 30, 40]
[20, 40]
[10, 30]
[1, 2, 4]
```

5.8 Insert an item at a specific index

Asked as: Sheet C7

```
let arr = [10, 20, 40, 50];
arr.splice(2, 0, 30);        // at index 2, delete 0, insert 30
console.log(arr);
```

Output:

```
[10, 20, 30, 40, 50]
```

5.9 Replace an item

Asked as: Sheet C13

```
let arr = [10, 20, 30, 40];
arr[1] = 25;                          // by index
let i = arr.indexOf(30);              // by value
if (i !== -1) arr[i] = 35;
console.log(arr);
console.log([10, 20, 30].map(x => x === 20 ? 99 : x));   // without changing original
```

Output:

```
[10, 25, 35, 40]
[10, 99, 30]
```

5.10 Reverse an array

Asked as: Sheet C4

```
let arr = [1, 2, 3, 4, 5];
let rev = [];
for (let i = arr.length - 1; i >= 0; i--) rev.push(arr[i]);   // manual
console.log(rev);
console.log([...arr].reverse());     // built-in on a copy (reverse() changes the original)
```

Output:

```
[5, 4, 3, 2, 1]
[5, 4, 3, 2, 1]
```

5.11 Empty an array

Asked as: Sheet C5

```
let arr = [1, 2, 3];
arr.length = 0;       // also: arr.splice(0)  or  arr = [] (reassign)
console.log(arr);
```

Output:

```
[]
```

5.12 Check whether an array includes a value

Asked as: Sheet C6

```
let arr = [10, 20, 30];
console.log(arr.includes(20));
console.log(arr.indexOf(99) !== -1);
console.log(arr.some(x => x > 25));     // by condition
```

Output:

```
true
false
true
```

5.13 Find the index of a value

Asked as: Sheet C32

```
let arr = [10, 20, 30, 20];
console.log(arr.indexOf(20));            // first
console.log(arr.lastIndexOf(20));        // last
console.log(arr.indexOf(99));            // not found
console.log(arr.findIndex(x => x > 15)); // by condition
```

Output:

```
1
3
-1
1
```

5.14 Sum of all elements (loop and reduce)

Asked as: Lab S5.3 · Sheet C9

```
let arr = [10, 20, 30, 40];
let sum = 0;
for (let i = 0; i < arr.length; i++) {
  sum += arr[i];
}
console.log("Sum (loop) =", sum);
console.log("Sum (reduce) =", arr.reduce((acc, x) => acc + x, 0));
```

Output:

```
Sum (loop) = 100
Sum (reduce) = 100
```

5.15 Check whether an object is an array

Asked as: Sheet C14

```
console.log(Array.isArray([1, 2]));
console.log(Array.isArray({ a: 1 }));
console.log(Array.isArray("abc"));
console.log(typeof [1, 2]);          // "object", so typeof can't tell
```

Output:

```
true
false
false
object
```

5.16 Select a random element

Asked as: Sheet C27

```
let colors = ["red", "green", "blue", "yellow"];
let pick = colors[Math.floor(Math.random() * colors.length)];
console.log(pick);
```

Output (random, so yours will differ):

```
red
```

5.17 Copy / clone an array (shallow vs deep)

Asked as: Sheet C31 · Sheet C37 · Theory M3-Q20

```
let a = [1, 2, 3];
let b = [...a];              // spread
let c = a.slice();           // slice
let d = Array.from(a);       // Array.from
let e = [];
e.push(...a);                // copy into an existing array
b.push(99);
console.log(a, b, c, d, e);
// shallow vs deep
let nested = [1, [2, 3]];
let shallow = [...nested];
let deep = structuredClone(nested);   // or JSON.parse(JSON.stringify(nested))
shallow[1][0] = 50;
console.log(nested[1][0], deep[1][0]);
```

Output:

```
[1, 2, 3] [1, 2, 3, 99] [1, 2, 3] [1, 2, 3] [1, 2, 3]
50 2
```

5.18 Comma-separated string to array

Asked as: Sheet C30 · Sheet D28

```
let str = "10, 20, 30, 40";
console.log(str.split(","));
console.log(str.split(",").map(s => s.trim()));
console.log(str.split(",").map(Number));
```

Output:

```
['10', ' 20', ' 30', ' 40']
['10', '20', '30', '40']
[10, 20, 30, 40]
```

6. Arrays: search, sort, duplicates, set operations

6.1 Sort a numeric array

Asked as: Sheet C17

```
let nums = [40, 5, 100, 1, 25];
console.log([...nums].sort());                  // WRONG: sorts as strings
console.log([...nums].sort((a, b) => a - b));   // ascending
console.log([...nums].sort((a, b) => b - a));   // descending
```

Output:

```
[1, 100, 25, 40, 5]
[1, 5, 25, 40, 100]
[100, 40, 25, 5, 1]
```

6.2 Sort student marks in descending order

Asked as: Lab S6.5

```
let students = [
  { name: "Amit", marks: 78 },
  { name: "Riya", marks: 92 },
  { name: "Kiran", marks: 85 }
];
students.sort((a, b) => b.marks - a.marks);
for (let s of students) console.log(s.name + " - " + s.marks);
let marks = [78, 92, 85, 60];
console.log(marks.sort((a, b) => b - a));
```

Output:

```
Riya - 92
Kiran - 85
Amit - 78
[92, 85, 78, 60]
```

6.3 Sort words alphabetically

Asked as: Sheet C10

```
let words = ["mango", "Apple", "banana", "Cherry"];
console.log([...words].sort());                                 // capitals first
console.log([...words].sort((a, b) => a.localeCompare(b)));     // proper A-Z
let sentence = "the quick brown fox";
console.log(sentence.split(" ").sort().join(" "));              // sort words in a string
```

Output:

```
['Apple', 'Cherry', 'banana', 'mango']
['Apple', 'banana', 'Cherry', 'mango']
brown fox quick the
```

6.4 Sort array of objects by a string property

Asked as: Sheet C22 · Theory M3-Q22

```
let books = [
  { title: "Zebra", year: 2001 },
  { title: "apple", year: 1999 },
  { title: "Mango", year: 2010 }
];
books.sort((a, b) => a.title.localeCompare(b.title));
console.log(books.map(b => b.title));
books.sort((a, b) => a.year - b.year);     // by number
console.log(books.map(b => b.year));
```

Output:

```
['apple', 'Mango', 'Zebra']
[1999, 2001, 2010]
```

6.5 Sort by multiple properties

Asked as: Theory M3-Q12

```
let s = [
  { name: "Riya", marks: 90 },
  { name: "Amit", marks: 90 },
  { name: "Kiran", marks: 85 }
];
// marks descending, then name ascending when marks are equal
s.sort((a, b) => b.marks - a.marks || a.name.localeCompare(b.name));
console.log(s.map(x => x.name + ":" + x.marks));
```

Output:

```
['Amit:90', 'Riya:90', 'Kiran:85']
```

6.6 Remove duplicates / unique values

Asked as: Sheet C11 · Sheet C18 · Sheet C19

```
let arr = [1, 2, 2, 3, 4, 4, 5];
console.log([...new Set(arr)]);                          // Set
console.log(arr.filter((v, i) => arr.indexOf(v) === i)); // filter + indexOf
let out = [];                                            // loop
for (let x of arr) {
  if (!out.includes(x)) out.push(x);
}
console.log(out);
```

Output:

```
[1, 2, 3, 4, 5]
[1, 2, 3, 4, 5]
[1, 2, 3, 4, 5]
```

6.7 Remove duplicates ignoring case

Asked as: Lab S6.3

```
let arr = ["Apple", "mango", "APPLE", "Banana", "MANGO"];
let seen = [];
let result = [];
for (let item of arr) {
  let lower = item.toLowerCase();
  if (!seen.includes(lower)) {
    seen.push(lower);
    result.push(item);        // keeps the first spelling
  }
}
console.log(result);
```

Output:

```
['Apple', 'mango', 'Banana']
```

6.8 Find duplicate elements

Asked as: Sheet C16

```
let arr = [1, 2, 3, 2, 4, 1, 5];
let seen = [], dup = [];
for (let x of arr) {
  if (seen.includes(x) && !dup.includes(x)) dup.push(x);
  seen.push(x);
}
console.log(dup);
```

Output:

```
[2, 1]
```

6.9 Filter the non-unique values

Asked as: Sheet C35

```
let arr = [1, 2, 2, 3, 4, 4, 5];
// keep only values that appear exactly once
console.log(arr.filter(x => arr.indexOf(x) === arr.lastIndexOf(x)));
```

Output:

```
[1, 3, 5]
```

6.10 Count occurrences of all items

Asked as: Sheet C34

```
let arr = ["a", "b", "a", "c", "b", "a"];
let count = {};
for (let x of arr) {
  count[x] = (count[x] || 0) + 1;
}
console.log(count);
```

Output:

```
{ a: 3, b: 2, c: 1 }
```

6.11 Most frequent element

Asked as: Lab S6.1 · Sheet C40 · Sheet C47

```
let arr = [3, "a", "a", "a", 2, 3, "a", 3, "a", 2, 4, 9, 3];
let count = {};
let maxItem = arr[0], maxCount = 0;
for (let item of arr) {
  count[item] = (count[item] || 0) + 1;
  if (count[item] > maxCount) {
    maxCount = count[item];
    maxItem = item;
  }
}
console.log(maxItem + " ( " + maxCount + " times )");
```

Output:

```
a ( 5 times )
```

6.12 Check if an array has only unique values

Asked as: Sheet C46

```
function allUnique(arr) {
  return new Set(arr).size === arr.length;
}
console.log(allUnique([1, 2, 3]), allUnique([1, 2, 2]));
```

Output:

```
true false
```

6.13 Common items / intersection of two arrays

Asked as: Sheet C12 · Sheet C28

```
let a = [1, 2, 3, 4, 5];
let b = [4, 5, 6, 7];
console.log("Intersection:", a.filter(x => b.includes(x)));
console.log("Difference  :", a.filter(x => !b.includes(x)));
console.log("Union       :", [...new Set([...a, ...b])]);
```

Output:

```
Intersection: [4, 5]
Difference  : [1, 2, 3]
Union       : [1, 2, 3, 4, 5, 6, 7]
```

6.14 Merge two arrays and remove duplicates

Asked as: Sheet C21 · Sheet C33

```
let a = [1, 2, 3];
let b = [3, 4, 5];
console.log([...new Set([...a, ...b])]);
console.log([...new Set(a.concat(b))]);
```

Output:

```
[1, 2, 3, 4, 5]
[1, 2, 3, 4, 5]
```

6.15 Compare two arrays

Asked as: Sheet C26

```
function areEqual(a, b) {
  if (a.length !== b.length) return false;
  for (let i = 0; i < a.length; i++) {
    if (a[i] !== b[i]) return false;
  }
  return true;
}
console.log(areEqual([1, 2, 3], [1, 2, 3]));
console.log(areEqual([1, 2, 3], [3, 2, 1]));
console.log([1, 2] === [1, 2]);   // false: arrays compare by reference
```

Output:

```
true
false
false
```

6.16 Get the N largest elements

Asked as: Sheet C36

```
function nLargest(arr, n) {
  return [...arr].sort((a, b) => b - a).slice(0, n);
}
console.log(nLargest([10, 45, 3, 78, 22, 90], 3));
```

Output:

```
[90, 78, 45]
```

7. Arrays: transform and algorithms

7.1 map(), filter(), reduce()

Asked as: Lab S6.4 · Theory M3-Q18

```
let nums = [1, 2, 3, 4, 5, 6];
let doubled = nums.map(x => x * 2);
let evens = nums.filter(x => x % 2 === 0);
let total = nums.reduce((sum, x) => sum + x, 0);
console.log("map    :", doubled);
console.log("filter :", evens);
console.log("reduce :", total);
console.log("chain  :", nums.filter(x => x > 2).map(x => x * 10));
```

Output:

```
map    : [2, 4, 6, 8, 10, 12]
filter : [2, 4, 6]
reduce : 21
chain  : [30, 40, 50, 60]
```

7.2 forEach, find, some, every

Asked as: Theory M3-Q4

```
let marks = [45, 72, 88, 39];
marks.forEach((m, i) => console.log("Student " + (i + 1) + ": " + m));
console.log("first above 80:", marks.find(m => m > 80));
console.log("any fail?     :", marks.some(m => m < 40));
console.log("all pass?     :", marks.every(m => m >= 40));
```

Output:

```
Student 1: 45
Student 2: 72
Student 3: 88
Student 4: 39
first above 80: 88
any fail?     : true
all pass?     : false
```

7.3 Extract a property as an array from objects

Asked as: Sheet C25

```
let students = [
  { name: "Amit", marks: 88 },
  { name: "Riya", marks: 92 },
  { name: "Kiran", marks: 75 }
];
console.log(students.map(s => s.name));
console.log(students.map(s => s.marks));
```

Output:

```
['Amit', 'Riya', 'Kiran']
[88, 92, 75]
```

7.4 Generate a range of numbers and characters

Asked as: Sheet C15

```
function range(start, end) {
  let out = [];
  for (let i = start; i <= end; i++) out.push(i);
  return out;
}
function charRange(from, to) {
  let out = [];
  for (let c = from.charCodeAt(0); c <= to.charCodeAt(0); c++) {
    out.push(String.fromCharCode(c));
  }
  return out;
}
console.log(range(1, 10));
console.log(charRange("A", "E"));
```

Output:

```
[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
['A', 'B', 'C', 'D', 'E']
```

7.5 Splice without mutating the original

Asked as: Sheet C24 · Theory M3-Q6

```
let arr = [1, 2, 3, 4, 5];
let copy = [...arr];
copy.splice(1, 2);                                  // splice the copy
console.log(arr, copy);
console.log(arr.slice(0, 1).concat(arr.slice(3)));  // slice + concat
console.log(arr.toSpliced(1, 2), arr);              // ES2023
```

Output:

```
[1, 2, 3, 4, 5] [1, 4, 5]
[1, 4, 5]
[1, 4, 5] [1, 2, 3, 4, 5]
```

7.6 Split an array into chunks

Asked as: Sheet C29

```
function chunk(arr, size) {
  let out = [];
  for (let i = 0; i < arr.length; i += size) {
    out.push(arr.slice(i, i + size));
  }
  return out;
}
console.log(chunk([1, 2, 3, 4, 5, 6, 7], 3));
```

Output:

```
[[1, 2, 3], [4, 5, 6], [7]]
```

7.7 Cumulative (running) sum

Asked as: Sheet C42

```
let arr = [1, 2, 3, 4, 5];
let out = [], sum = 0;
for (let x of arr) {
  sum += x;
  out.push(sum);
}
console.log(out);
```

Output:

```
[1, 3, 6, 10, 15]
```

7.8 Merge two arrays without creating a new array

Asked as: Sheet C43

```
let a = [1, 2, 3];
let b = [4, 5, 6];
for (let x of b) a.push(x);     // or a.push(...b)
console.log(a);
```

Output:

```
[1, 2, 3, 4, 5, 6]
```

7.9 Right rotate an array r times

Asked as: Lab S5.5

```
function rotateRight(a, r) {
  r = r % a.length;
  return a.slice(a.length - r).concat(a.slice(0, a.length - r));
}
console.log(rotateRight([1, 2, 3, 4, 5], 2));
console.log(rotateRight([1, 2, 3, 4, 5], 7));    // 7 % 5 = 2
```

Output:

```
[4, 5, 1, 2, 3]
[4, 5, 1, 2, 3]
```

7.10 Left rotate by one (and by d)

Asked as: Sheet C48

```
function leftRotateOne(arr) {
  let first = arr[0];
  for (let i = 0; i < arr.length - 1; i++) arr[i] = arr[i + 1];
  arr[arr.length - 1] = first;
  return arr;
}
function leftRotate(arr, d) {
  d = d % arr.length;
  return arr.slice(d).concat(arr.slice(0, d));
}
console.log(leftRotateOne([1, 2, 3, 4, 5]));
console.log(leftRotate([1, 2, 3, 4, 5], 2));
```

Output:

```
[2, 3, 4, 5, 1]
[3, 4, 5, 1, 2]
```

7.11 Rotation count of a rotated sorted array

Asked as: Sheet C49

```
// rotation count = index of the smallest element
function rotationCount(arr) {
  let low = 0, high = arr.length - 1;
  while (low < high) {                  // binary search, O(log n)
    let mid = Math.floor((low + high) / 2);
    if (arr[mid] > arr[high]) low = mid + 1;
    else high = mid;
  }
  return low;
}
console.log(rotationCount([15, 18, 2, 3, 6, 12]));
console.log(rotationCount([7, 9, 11, 12, 5]));
console.log(rotationCount([1, 2, 3]));
```

Output:

```
2
4
0
```

7.12 Shuffle a deck of cards

Asked as: Sheet B11

```
let suits = ["Hearts", "Diamonds", "Clubs", "Spades"];
let ranks = ["A", "2", "3", "4", "5", "6", "7", "8", "9", "10", "J", "Q", "K"];
let deck = [];
for (let s of suits) {
  for (let r of ranks) deck.push(r + " of " + s);
}
// Fisher-Yates shuffle
for (let i = deck.length - 1; i > 0; i--) {
  let j = Math.floor(Math.random() * (i + 1));
  let temp = deck[i];
  deck[i] = deck[j];
  deck[j] = temp;
}
console.log(deck.length, "cards. Top 5:", deck.slice(0, 5));
```

Output (random, so yours will differ):

```
52 cards. Top 5: ['2 of Diamonds', 'J of Clubs', '8 of Hearts', '9 of Diamonds', '3 of Hearts']
```

7.13 Smallest difference triplet from three arrays

Asked as: Sheet C38

```
function smallestDiffTriplet(a, b, c) {
  a.sort((x, y) => x - y);
  b.sort((x, y) => x - y);
  c.sort((x, y) => x - y);
  let i = 0, j = 0, k = 0, best = Infinity, result = [];
  while (i < a.length && j < b.length && k < c.length) {
    let max = Math.max(a[i], b[j], c[k]);
    let min = Math.min(a[i], b[j], c[k]);
    if (max - min < best) {
      best = max - min;
      result = [max, a[i] + b[j] + c[k] - max - min, min];
    }
    if (min === a[i]) i++;          // move the pointer at the minimum
    else if (min === b[j]) j++;
    else k++;
  }
  return result;
}
console.log(smallestDiffTriplet([5, 2, 8], [10, 7, 12], [9, 14, 6]));
```

Output:

```
[7, 6, 5]
```

7.14 Shortest distance between two words

Asked as: Sheet C39

```
function shortestDistance(words, w1, w2) {
  let p1 = -1, p2 = -1, min = Infinity;
  for (let i = 0; i < words.length; i++) {
    if (words[i] === w1) p1 = i;
    if (words[i] === w2) p2 = i;
    if (p1 !== -1 && p2 !== -1) min = Math.min(min, Math.abs(p1 - p2));
  }
  return min === Infinity ? -1 : min;
}
let words = ["the", "quick", "brown", "fox", "quick", "the"];
console.log(shortestDistance(words, "the", "fox"));
```

Output:

```
2
```

7.15 Max profit buying and selling a share at most twice

Asked as: Sheet C41

```
function maxProfitTwice(prices) {
  let buy1 = -Infinity, sell1 = 0, buy2 = -Infinity, sell2 = 0;
  for (let p of prices) {
    buy1 = Math.max(buy1, -p);          // best after 1st buy
    sell1 = Math.max(sell1, buy1 + p);  // best after 1st sell
    buy2 = Math.max(buy2, sell1 - p);   // best after 2nd buy
    sell2 = Math.max(sell2, buy2 + p);  // best after 2nd sell
  }
  return sell2;
}
console.log(maxProfitTwice([10, 22, 5, 75, 65, 80]));   // 12 + 75
console.log(maxProfitTwice([2, 30, 15, 10, 8, 25, 80])); // 28 + 72
```

Output:

```
87
100
```

7.16 K-th largest sum of a contiguous sub-array

Asked as: Sheet C44

```
function kthLargestSum(arr, k) {
  let sums = [];
  for (let i = 0; i < arr.length; i++) {
    let sum = 0;
    for (let j = i; j < arr.length; j++) {
      sum += arr[j];
      sums.push(sum);
    }
  }
  sums.sort((a, b) => b - a);
  return sums[k - 1];
}
console.log(kthLargestSum([20, -5, -1], 3));
console.log(kthLargestSum([10, -10, 20, -40], 6));
```

Output:

```
14
-10
```

7.17 Construct an array from its pair-sum array

Asked as: Sheet C45

```
// pair = [a0+a1, a0+a2, ..., a0+a(n-1), a1+a2, a1+a3, ...]
function fromPairSum(pair, n) {
  let arr = [];
  arr[0] = (pair[0] + pair[1] - pair[n - 1]) / 2;   // pair[n-1] = a1 + a2
  for (let i = 1; i < n; i++) arr[i] = pair[i - 1] - arr[0];
  return arr;
}
console.log(fromPairSum([15, 13, 11, 10, 12, 10, 9, 8, 7, 5], 5));
```

Output:

```
[8, 7, 5, 3, 2]
```

8. 2D Arrays (Matrices)

8.1 Create and access a 2D array

Asked as: Lab S6.6 · Sheet C23

```
let matrix = [
  [1, 2, 3],
  [4, 5, 6],
  [7, 8, 9]
];
console.log("Element [1][2] =", matrix[1][2]);
console.log("Rows:", matrix.length, "Cols:", matrix[0].length);
for (let i = 0; i < matrix.length; i++) {
  let row = "";
  for (let j = 0; j < matrix[i].length; j++) row += matrix[i][j] + " ";
  console.log(row);
}
```

Output:

```
Element [1][2] = 6
Rows: 3 Cols: 3
1 2 3
4 5 6
7 8 9
```

8.2 Create an r x c matrix dynamically (and the fill trap)

Asked as: Sheet C23

```
let r = 3, c = 4;
let grid = [];
for (let i = 0; i < r; i++) {
  grid[i] = [];
  for (let j = 0; j < c; j++) grid[i][j] = i * c + j;
}
console.log(grid);
// WRONG: every row is the SAME array
let bad = new Array(2).fill(new Array(2).fill(0));
bad[0][0] = 5;
console.log("bad:", bad);
// RIGHT one-liner
let good = Array.from({ length: 2 }, () => new Array(2).fill(0));
good[0][0] = 5;
console.log("good:", good);
```

Output:

```
[[0, 1, 2, 3], [4, 5, 6, 7], [8, 9, 10, 11]]
bad: [[5, 0], [5, 0]]
good: [[5, 0], [0, 0]]
```

8.3 Row sums, column sums, total

Asked as: Lab S6.6

```
let m = [[1, 2, 3], [4, 5, 6], [7, 8, 9]];
let total = 0;
for (let i = 0; i < m.length; i++) {
  let rowSum = 0;
  for (let j = 0; j < m[i].length; j++) {
    rowSum += m[i][j];
    total += m[i][j];
  }
  console.log("Row " + i + " sum = " + rowSum);
}
for (let j = 0; j < m[0].length; j++) {
  let colSum = 0;
  for (let i = 0; i < m.length; i++) colSum += m[i][j];
  console.log("Col " + j + " sum = " + colSum);
}
console.log("Total =", total);
```

Output:

```
Row 0 sum = 6
Row 1 sum = 15
Row 2 sum = 24
Col 0 sum = 12
Col 1 sum = 15
Col 2 sum = 18
Total = 45
```

8.4 Matrix addition, transpose, multiplication

Asked as: Lab S6.6

```
let A = [[1, 2], [3, 4]];
let B = [[5, 6], [7, 8]];
function add(a, b) {
  return a.map((row, i) => row.map((v, j) => v + b[i][j]));
}
function transpose(a) {
  let t = [];
  for (let j = 0; j < a[0].length; j++) {
    t[j] = [];
    for (let i = 0; i < a.length; i++) t[j][i] = a[i][j];
  }
  return t;
}
function multiply(a, b) {
  let res = [];
  for (let i = 0; i < a.length; i++) {
    res[i] = [];
    for (let j = 0; j < b[0].length; j++) {
      let s = 0;
      for (let k = 0; k < b.length; k++) s += a[i][k] * b[k][j];
      res[i][j] = s;
    }
  }
  return res;
}
console.log("A+B :", add(A, B));
console.log("A^T :", transpose(A));
console.log("AxB :", multiply(A, B));
```

Output:

```
A+B : [[6, 8], [10, 12]]
A^T : [[1, 3], [2, 4]]
AxB : [[19, 22], [43, 50]]
```

8.5 Jagged array and flatten

Asked as: Theory M3-Q21

```
let jag = [[1], [2, 3], [4, 5, 6]];
for (let row of jag) console.log(row.join(" "));
console.log(jag.flat());
```

Output:

```
1
2 3
4 5 6
[1, 2, 3, 4, 5, 6]
```

9. Strings

9.1 Common string methods (quick reference)

Asked as: Theory M5-Q10

```
let s = "  Hello World  ";
console.log("[" + s.trim() + "]", s.length, s.trim().length);
let t = "Hello World";
console.log(t.toUpperCase(), "|", t.toLowerCase());
console.log(t.charAt(4), t[4], t.indexOf("o"), t.lastIndexOf("o"));
console.log(t.slice(0, 5), "|", t.substring(6), "|", t.slice(-5));
console.log(t.includes("World"), t.startsWith("He"), t.endsWith("d"));
console.log(t.split(" "), t.repeat(2), "7".padStart(3, "0"));
```

Output:

```
[Hello World] 15 11
HELLO WORLD | hello world
o o 4 7
Hello | World | World
true true true
['Hello', 'World'] Hello WorldHello World 007
```

9.2 Count words in a string

Asked as: Sheet D1

```
function countWords(str) {
  let words = str.trim().split(/\s+/);    // split on one or more spaces
  return str.trim() === "" ? 0 : words.length;
}
console.log(countWords("  The quick   brown fox  "));
console.log(countWords(""));
```

Output:

```
4
0
```

9.3 Check if a string contains whitespace

Asked as: Sheet D2

```
function hasWhitespace(str) {
  return /\s/.test(str);
}
console.log(hasWhitespace("Hello World"), hasWhitespace("HelloWorld"));
```

Output:

```
true false
```

9.4 Reverse a string

Asked as: Sheet D3

```
let str = "hello";
console.log(str.split("").reverse().join(""));
let rev = "";                                    // manual loop
for (let i = str.length - 1; i >= 0; i--) rev += str[i];
console.log(rev);
```

Output:

```
olleh
olleh
```

9.5 Replace all occurrences / replace characters / replace all dots

Asked as: Sheet D4 · Sheet D15 · Sheet D17 · Sheet D37

```
let s = "I like cats. cats are cute.";
console.log(s.replaceAll("cats", "dogs"));       // ES2021
console.log(s.replace(/cats/g, "dogs"));          // regex with g flag
console.log(s.split("cats").join("dogs"));        // split + join
console.log("hello".replace("l", "L"));           // only the FIRST match
console.log("1.234.567".replaceAll(".", ","));    // replace all dots
console.log("a-b_c".replace(/[-_]/g, " "));       // several characters at once
```

Output:

```
I like dogs. dogs are cute.
I like dogs. dogs are cute.
I like dogs. dogs are cute.
heLlo
1,234,567
a b c
```

9.6 Starts with / contains a substring

Asked as: Sheet D5 · Sheet D12 · Sheet D33

```
let s = "JavaScript is fun";
console.log(s.startsWith("Java"), s.endsWith("fun"));
console.log(s.includes("Script"), s.indexOf("Script") !== -1);
console.log(s.includes("script"));                    // case-sensitive
console.log(s.toLowerCase().includes("script"));      // case-insensitive
```

Output:

```
true true
true true
false
true
```

9.7 Multi-line strings

Asked as: Sheet D6

```
let a = `Line 1
Line 2
Line 3`;                                  // template literal
let b = "Line 1\nLine 2\nLine 3";         // \n escape
console.log(a);
console.log(a === b);
```

Output:

```
Line 1
Line 2
Line 3
true
```

9.8 Format a number as a currency string

Asked as: Sheet D7

```
let amount = 1234567.891;
console.log(amount.toLocaleString("en-IN", { style: "currency", currency: "INR" }));
console.log(amount.toLocaleString("en-US", { style: "currency", currency: "USD" }));
console.log("Rs. " + amount.toFixed(2));
```

Output:

```
₹12,34,567.89
$1,234,567.89
Rs. 1234567.89
```

9.9 Trim a string at the beginning or end

Asked as: Sheet D8

```
let s = "   hello   ";
console.log("[" + s.trim() + "]");
console.log("[" + s.trimStart() + "]");
console.log("[" + s.trimEnd() + "]");
```

Output:

```
[hello]
[hello   ]
[   hello]
```

9.10 Delete the first character / get the last character

Asked as: Sheet D9 · Sheet D10

```
let s = "Hello";
console.log(s.slice(1));            // delete first char
console.log(s.substring(1));
console.log(s[s.length - 1]);       // last char
console.log(s.charAt(s.length - 1), s.slice(-1), s.at(-1));
```

Output:

```
ello
ello
o
o o o
```

9.11 Convert an object to a string

Asked as: Sheet D11

```
let obj = { name: "Amit", age: 21 };
console.log(JSON.stringify(obj));
console.log(String(obj));           // not useful: "[object Object]"
let parts = [];
for (let k in obj) parts.push(k + "=" + obj[k]);
console.log(parts.join(", "));
```

Output:

```
{"name":"Amit","age":21}
[object Object]
name=Amit, age=21
```

9.12 Compare strings (and case-insensitively)

Asked as: Sheet D13 · Sheet D24

```
let a = "apple", b = "Apple";
console.log(a === b);                                   // false, case matters
console.log(a.toLowerCase() === b.toLowerCase());       // true
console.log("apple".localeCompare("banana"));           // -1 means a comes first
console.log("b" > "a", "B" > "a");                      // compares character codes
console.log(a.localeCompare(b, undefined, { sensitivity: "base" }) === 0);
```

Output:

```
false
true
-1
true false
true
```

9.13 Encode / decode a string to Base64

Asked as: Sheet D14

```
let text = "Hello JS";
let encoded = btoa(text);            // in the browser
let decoded = atob(encoded);
console.log(encoded, "->", decoded);
```

Output:

```
SGVsbG8gSlM= -> Hello JS
```

9.14 Palindrome check

Asked as: Sheet D16

```
function isPalindrome(str) {
  let clean = str.toLowerCase().replace(/[^a-z0-9]/g, "");
  return clean === clean.split("").reverse().join("");
}
console.log(isPalindrome("Madam"));
console.log(isPalindrome("A man, a plan, a canal: Panama"));
console.log(isPalindrome("hello"));
```

Output:

```
true
true
false
```

9.15 Count occurrences of each letter / character

Asked as: Sheet D18 · Sheet D42

```
function charCount(str) {
  let count = {};
  for (let ch of str) {
    if (ch === " ") continue;
    count[ch] = (count[ch] || 0) + 1;
  }
  return count;
}
console.log(charCount("hello world"));
```

Output:

```
{ h: 1, e: 1, l: 3, o: 2, w: 1, r: 1, d: 1 }
```

9.16 Generate a random string

Asked as: Sheet D19

```
function randomString(len) {
  let chars = "ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789";
  let out = "";
  for (let i = 0; i < len; i++) {
    out += chars[Math.floor(Math.random() * chars.length)];
  }
  return out;
}
console.log(randomString(8));
```

Output (random, so yours will differ):

```
1jo20hPQ
```

9.17 First letter uppercase / title case

Asked as: Sheet D20 · Sheet D30

```
function capitalize(s) {
  return s.charAt(0).toUpperCase() + s.slice(1);
}
function titleCase(s) {
  return s.toLowerCase().split(" ").map(w => capitalize(w)).join(" ");
}
console.log(capitalize("javascript"));
console.log(titleCase("the QUICK brown fox"));
```

Output:

```
Javascript
The Quick Brown Fox
```

9.18 Count vowels

Asked as: Sheet D21

```
function countVowels(str) {
  let count = 0;
  for (let ch of str.toLowerCase()) {
    if ("aeiou".includes(ch)) count++;
  }
  return count;
}
console.log(countVowels("Front End Design"));
console.log(("Front End Design".match(/[aeiou]/gi) || []).length);   // regex way
```

Output:

```
4
4
```

9.19 Remove spaces from a string

Asked as: Sheet D22

```
let s = "  J a v a  Script ";
console.log(s.replace(/\s/g, ""));
console.log(s.split(" ").join(""));
```

Output:

```
JavaScript
JavaScript
```

9.20 Count how many times a string occurs in a string

Asked as: Sheet D23

```
function countOccurrences(text, word) {
  let count = 0, pos = text.indexOf(word);
  while (pos !== -1) {
    count++;
    pos = text.indexOf(word, pos + word.length);
  }
  return count;
}
let t = "cat and cat and another cat";
console.log(countOccurrences(t, "cat"));
console.log(t.split("cat").length - 1);      // shortcut
```

Output:

```
3
3
```

9.21 All combinations (subsets) and permutations of a string

Asked as: Sheet D25

```
function combinations(str) {          // every subset, order kept
  let result = [];
  let n = str.length;
  for (let mask = 1; mask < (1 << n); mask++) {
    let combo = "";
    for (let i = 0; i < n; i++) {
      if (mask & (1 << i)) combo += str[i];
    }
    result.push(combo);
  }
  return result;
}
function permutations(str) {          // every ordering
  if (str.length <= 1) return [str];
  let result = [];
  for (let i = 0; i < str.length; i++) {
    let rest = str.slice(0, i) + str.slice(i + 1);
    for (let p of permutations(rest)) result.push(str[i] + p);
  }
  return result;
}
console.log(combinations("abc"));
console.log(permutations("abc"));
```

Output:

```
['a', 'b', 'ab', 'c', 'ac', 'bc', 'abc']
['abc', 'acb', 'bac', 'bca', 'cab', 'cba']
```

9.22 Unique characters of a string

Asked as: Sheet D26

```
let s = "programming";
console.log([...new Set(s)].join(""));                        // remove repeats
console.log(s.split("").filter(c => s.indexOf(c) === s.lastIndexOf(c)).join(""));  // appear once only
```

Output:

```
progamin
poain
```

9.23 String to float / strip non-numeric characters

Asked as: Sheet D29 · Sheet D32

```
console.log(parseFloat("3.14abc"), Number("3.14"), +"2.5");
console.log("Rs. 1,299.50 only".replace(/[^0-9.]/g, ""));     // keep digits and dot
console.log("ph: 98-76-543210".replace(/\D/g, ""));           // digits only
```

Output:

```
3.14 3.14 2.5
.1299.50
9876543210
```

9.24 Remove non-ASCII characters

Asked as: Sheet D31

```
let s = "Café naïve — résumé ✓";
console.log(s.replace(/[^\x00-\x7F]/g, ""));
```

Output:

```
Caf nave  rsum
```

9.25 Split a multiline string into lines

Asked as: Sheet D34

```
let text = "first line\nsecond line\r\nthird line";
console.log(text.split(/\r?\n/));
```

Output:

```
['first line', 'second line', 'third line']
```

9.26 Longest word / smallest and largest word

Asked as: Sheet D35 · Sheet D44

```
function wordExtremes(str) {
  let words = str.split(/\s+/).filter(w => w !== "");
  let smallest = words[0], largest = words[0];
  for (let w of words) {
    if (w.length < smallest.length) smallest = w;
    if (w.length > largest.length) largest = w;
  }
  return { smallest, largest };
}
console.log(wordExtremes("JavaScript is a powerful language"));
```

Output:

```
{ smallest: 'a', largest: 'JavaScript' }
```

9.27 Nth occurrence of a string

Asked as: Sheet D36

```
function nthIndex(text, word, n) {
  let pos = -1;
  for (let i = 0; i < n; i++) {
    pos = text.indexOf(word, pos + 1);
    if (pos === -1) return -1;
  }
  return pos;
}
let t = "one two one three one";
console.log(nthIndex(t, "one", 2), nthIndex(t, "one", 3), nthIndex(t, "one", 4));
```

Output:

```
8 18 -1
```

9.28 Anagram check

Asked as: Sheet D38

```
function isAnagram(a, b) {
  let clean = s => s.toLowerCase().replace(/\s/g, "").split("").sort().join("");
  return clean(a) === clean(b);
}
console.log(isAnagram("Listen", "Silent"));
console.log(isAnagram("hello", "world"));
```

Output:

```
true
false
```

9.29 Minimum changes to make two strings anagrams (no deletion)

Asked as: Sheet D45

```
function minManipulations(s1, s2) {
  let count = {};
  for (let ch of s1) count[ch] = (count[ch] || 0) + 1;
  for (let ch of s2) count[ch] = (count[ch] || 0) - 1;
  let diff = 0;
  for (let k in count) diff += Math.abs(count[k]);
  return diff / 2;
}
console.log(minManipulations("ddcf", "cedk"));
console.log(minManipulations("aba", "baa"));
```

Output:

```
2
0
```

9.30 Does a string follow the character order of a pattern?

Asked as: Sheet D39

```
// every char of pattern[i] must appear before every char of pattern[i+1]
function followsOrder(str, pattern) {
  for (let i = 0; i < pattern.length - 1; i++) {
    let lastOfThis = str.lastIndexOf(pattern[i]);
    let firstOfNext = str.indexOf(pattern[i + 1]);
    if (lastOfThis === -1 || firstOfNext === -1 || lastOfThis > firstOfNext) return false;
  }
  return true;
}
console.log(followsOrder("engineers rock", "er"));
console.log(followsOrder("engineers rock", "gsr"));
console.log(followsOrder("engineers rock", "egr"));
```

Output:

```
true
false
false
```

9.31 Check if a string contains a digit

Asked as: Sheet D41

```
console.log(/\d/.test("abc123"), /\d/.test("abcdef"));
let hasDigit = false;
for (let ch of "abc1") if (ch >= "0" && ch <= "9") hasDigit = true;
console.log(hasDigit);
```

Output:

```
true false
true
```

9.32 i-th character of a binary string after n iterations

Asked as: Sheet D43

```
// convert m to binary; each iteration replaces 0 -> "01" and 1 -> "10"
function ithChar(m, n, i) {
  let s = m.toString(2);
  for (let k = 0; k < n; k++) {
    let next = "";
    for (let ch of s) next += (ch === "0") ? "01" : "10";
    s = next;
  }
  return s[i];
}
console.log(ithChar(5, 2, 5));    // "101" -> "100110" -> "100101101001", index 5
console.log(ithChar(11, 1, 3));   // "1011" -> "10011010", index 3
```

Output:

```
1
1
```

9.33 All substrings of a string

Asked as: Sheet D46

```
function substrings(str) {
  let result = [];
  for (let i = 0; i < str.length; i++) {
    for (let j = i + 1; j <= str.length; j++) result.push(str.slice(i, j));
  }
  return result;
}
console.log(substrings("abc"));
```

Output:

```
['a', 'ab', 'abc', 'b', 'bc', 'c']
```

9.34 Swap the case of each character

Asked as: Theory M5-Q15

```
function swapCase(str) {
  let out = "";
  for (let ch of str) {
    out += (ch === ch.toUpperCase()) ? ch.toLowerCase() : ch.toUpperCase();
  }
  return out;
}
console.log(swapCase("Hello World 123"));
```

Output:

```
hELLO wORLD 123
```

10. Regular Expressions and Validation

10.1 Regex cheat sheet with tests

Asked as: Theory M5-Q14

```
// ^ start  $ end  \d digit  \w word char  \s space  . any char
// + one or more  * zero or more  ? optional  {n,m} n to m times
// [abc] one of  [^abc] none of  (x|y) x or y   flags: g global, i ignore case
console.log(/^\d{6}$/.test("834001"));          // 6-digit PIN
console.log(/^\d{6}$/.test("83400"));
console.log(/^[A-Z]/.test("Hello"));            // starts with capital
console.log(/cat/i.test("CAT"));                // ignore case
console.log("a1b22c333".match(/\d+/g));         // all number groups
console.log(/(\d+)-(\d+)/.exec("10-20"));       // exec gives groups
```

Output:

```
true
false
true
true
['1', '22', '333']
['10-20', '10', '20']
```

10.2 Validate an email with RegExp (test())

Asked as: Lab S10.4

```
let pattern = /^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$/;
let tests = ["amit@gmail.com", "amit.k@bitmesra.ac.in", "amit@gmail", "amit gmail.com", "@x.com"];
for (let e of tests) {
  console.log(e.padEnd(24), pattern.test(e) ? "valid" : "invalid");
}
// pattern design: name part + @ + domain + . + 2+ letters, anchored with ^ and $
```

Output:

```
amit@gmail.com           valid
amit.k@bitmesra.ac.in    valid
amit@gmail               invalid
amit gmail.com           invalid
@x.com                   invalid
```

10.3 Common validation patterns

Asked as: Lab S9.2

```
let rules = {
  name: /^[A-Za-z ]{2,30}$/,
  mobile: /^[6-9]\d{9}$/,
  password: /^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*[@$!%*?&]).{8,}$/,
  pincode: /^[1-9]\d{5}$/,
  usn: /^[1-4][A-Z]{2}\d{2}[A-Z]{2}\d{3}$/
};
let values = { name: "Amit K", mobile: "9876543210", password: "Abcd@123",
               pincode: "834001", usn: "1BM20CS001" };
for (let field in rules) {
  console.log(field.padEnd(9), values[field].padEnd(12), rules[field].test(values[field]));
}
console.log(rules.password.test("abcd1234"));   // no capital, no special char
```

Output:

```
name      Amit K       true
mobile    9876543210   true
password  Abcd@123     true
pincode   834001       true
usn       1BM20CS001   true
false
```

10.4 Convert user input into a RegExp

Asked as: Sheet D27

```
let userInput = "c.t";                      // e.g. from prompt()
let re1 = new RegExp(userInput, "gi");      // treats . as "any char"
console.log("cat cot c.t".match(re1));
let escaped = userInput.replace(/[.*+?^${}()|[\]\\]/g, "\\$&");   // literal match
let re2 = new RegExp(escaped, "g");
console.log("cat cot c.t".match(re2));
```

Output:

```
['cat', 'cot', 'c.t']
['c.t']
```

10.5 Extract email addresses from text

Asked as: Sheet D40

```
let text = "Contact amit@gmail.com or hr@bitmesra.ac.in, not @invalid.";
let emails = text.match(/[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}/g);
console.log(emails);
```

Output:

```
['amit@gmail.com', 'hr@bitmesra.ac.in']
```

10.6 Complete form validation page (HTML + JS)

Asked as: Lab S9.2 · Sec B-1 · Theory M2-Q11

```
<!DOCTYPE html>
<html>
<head>
<style>
  body { font-family: Arial; width: 360px; margin: 30px auto; }
  input { display: block; width: 100%; padding: 6px; margin-top: 4px; }
  .err { color: red; font-size: 13px; }
</style>
</head>
<body>
<h3>Register</h3>
<form onsubmit="return validate()">
  Name <input id="name"> <span class="err" id="nameErr"></span>
  Email <input id="email"> <span class="err" id="emailErr"></span>
  Password <input type="password" id="pwd"> <span class="err" id="pwdErr"></span>
  Confirm <input type="password" id="cpwd"> <span class="err" id="cpwdErr"></span>
  Mobile <input id="phone"> <span class="err" id="phoneErr"></span>
  <br><input type="submit" value="Register">
</form>
<p id="msg"></p>
<script>
function validate() {
  let ok = true;
  function check(id, condition, message) {
    let errBox = document.getElementById(id + "Err");
    if (!condition) { errBox.innerHTML = message; ok = false; }
    else errBox.innerHTML = "";
  }
  let name = document.getElementById("name").value.trim();
  let email = document.getElementById("email").value.trim();
  let pwd = document.getElementById("pwd").value;
  let cpwd = document.getElementById("cpwd").value;
  let phone = document.getElementById("phone").value.trim();

  check("name", /^[A-Za-z ]{2,30}$/.test(name), "Letters only, 2-30 chars");
  check("email", /^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$/.test(email), "Invalid email");
  check("pwd", /^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*[@$!%*?&]).{8,}$/.test(pwd),
        "8+ chars with upper, lower, digit, special");
  check("cpwd", pwd === cpwd && cpwd !== "", "Passwords do not match");
  check("phone", /^[6-9]\d{9}$/.test(phone), "10-digit mobile starting 6-9");

  if (ok) document.getElementById("msg").innerHTML = "Registered successfully!";
  return false;          // stop page reload (return ok to really submit)
}
</script>
</body>
</html>
```

Output: a red error message appears beside each invalid field. When every field is valid, *Registered successfully!* is shown.

10.7 USN and semester validation (Section C Q1 a and b)

Asked as: Sec C-1a · Sec C-1b

```
<!DOCTYPE html>
<html>
<body>
<h3>Student Details</h3>
<form onsubmit="return validateForm()">
  USN: <input type="text" id="usn" onchange="checkUSN()"><br><br>
  Semester: <input type="text" id="sem" onchange="checkSem()"><br><br>
  <input type="submit" value="Submit">
</form>
<script>
// a) digit 1-4, two capitals, two digits, two capitals, three digits, no spaces
function checkUSN() {
  let usn = document.getElementById("usn").value;
  let pattern = /^[1-4][A-Z]{2}[0-9]{2}[A-Z]{2}[0-9]{3}$/;
  if (!pattern.test(usn)) {
    alert("Invalid USN! Format: 1BM20CS001 (no spaces)");
    return false;
  }
  return true;
}
// b) semester must be a number from 1 to 8
function checkSem() {
  let sem = document.getElementById("sem").value;
  if (!/^[1-8]$/.test(sem)) {
    alert("Invalid semester! Enter a number from 1 to 8");
    return false;
  }
  return true;
}
function validateForm() {
  if (checkUSN() && checkSem()) {
    alert("USN and semester are valid");
  }
  return false;
}
</script>
</body>
</html>
```

Output: `1BM20CS001` with semester `5` gives *USN and semester are valid*. `1bm20cs001` gives *Invalid USN!* (lowercase), and semester `9` gives *Invalid semester!*

```
let usnPattern = /^[1-4][A-Z]{2}[0-9]{2}[A-Z]{2}[0-9]{3}$/;
for (let u of ["1BM20CS001", "5BM20CS001", "1bm20cs001", "1BM 20CS001"]) console.log(u, usnPattern.test(u));
for (let s of ["1", "8", "9", "0", "10"]) console.log("sem", s, /^[1-8]$/.test(s));
```

Output:

```
1BM20CS001 true
5BM20CS001 false
1bm20cs001 false
1BM 20CS001 false
sem 1 true
sem 8 true
sem 9 false
sem 0 false
sem 10 false
```

11. Objects

11.1 Create and access object properties

Asked as: Lab S3.1 · Theory M2-Q1

```
let student = {
  name: "Riya",
  roll: 23,
  branch: "CSE",
  greet: function () { return "Hi, I am " + this.name; }
};
console.log(student.name);          // dot notation
console.log(student["branch"]);     // bracket notation
let key = "roll";
console.log(student[key]);          // bracket needed when key is in a variable
student.cgpa = 8.7;                 // add
student.roll = 24;                  // update
delete student.branch;              // delete
console.log(student.greet(), "has cgpa?", "cgpa" in student);
console.log(student);
```

Output:

```
Riya
CSE
23
Hi, I am Riya has cgpa? true
{ name: 'Riya', roll: 24, greet: [Function: greet], cgpa: 8.7 }
```

11.2 Ways to create an object

Asked as: Theory M2-Q1 · Theory M3-Q17

```
let o1 = { a: 1 };                                  // 1. literal
let o2 = new Object(); o2.a = 1;                    // 2. new Object()
function Car(brand) { this.brand = brand; }         // 3. constructor function
let o3 = new Car("Honda");
let o4 = Object.create({ hello() { return "hi"; } }); // 4. Object.create (prototype)
class Bike { constructor(b) { this.brand = b; } }   // 5. class
let o5 = new Bike("Yamaha");
console.log(o1, o2, o3, o4.hello(), o5);
```

Output:

```
{ a: 1 } { a: 1 } Car { brand: 'Honda' } hi Bike { brand: 'Yamaha' }
```

11.3 Accessors and mutators (get / set)

Asked as: Lab S3.2

```
let person = {
  firstName: "Amit",
  lastName: "Kumar",
  _age: 20,
  get fullName() {                          // accessor
    return this.firstName + " " + this.lastName;
  },
  set fullName(value) {                     // mutator
    let parts = value.split(" ");
    this.firstName = parts[0];
    this.lastName = parts[1];
  },
  get age() { return this._age; },
  set age(v) {
    if (v < 0) console.log("Age cannot be negative");
    else this._age = v;
  }
};
console.log(person.fullName);
person.fullName = "Rahul Sharma";
console.log(person.firstName, person.lastName);
person.age = -5;
person.age = 22;
console.log(person.age);
```

Output:

```
Amit Kumar
Rahul Sharma
Age cannot be negative
22
```

11.4 Iterate object properties with for-in (and Object.keys/values/entries)

Asked as: Lab S3.3

```
let car = { brand: "Honda", model: "City", year: 2022 };
for (let key in car) {
  console.log(key + " : " + car[key]);
}
console.log(Object.keys(car));
console.log(Object.values(car));
console.log(Object.entries(car));
```

Output:

```
brand : Honda
model : City
year : 2022
['brand', 'model', 'year']
['Honda', 'City', 2022]
[['brand', 'Honda'], ['model', 'City'], ['year', 2022]]
```

11.5 Static methods and nullifying an object

Asked as: Lab S3.4

```
class MathUtil {
  static square(x) { return x * x; }       // belongs to the class
  static PI = 3.14159;                     // static property
}
console.log("Square of 5 =", MathUtil.square(5), "| PI =", MathUtil.PI);
let m = new MathUtil();
console.log("On an object:", typeof m.square);   // undefined, static is not on objects

let obj = { id: 1, name: "Test" };
console.log("Before:", obj);
obj = null;               // reference removed -> eligible for garbage collection
console.log("After:", obj);
```

Output:

```
Square of 5 = 25 | PI = 3.14159
On an object: undefined
Before: { id: 1, name: 'Test' }
After: null
```

11.6 Prototype, freeze and seal

Asked as: Theory M2-Q4

```
function Animal(name) { this.name = name; }
Animal.prototype.speak = function () { return this.name + " makes a sound"; };
let a = new Animal("Cat");
console.log(a.speak(), "| own property?", a.hasOwnProperty("speak"));

let cfg = Object.freeze({ url: "api.com" });
cfg.url = "hack.com";              // ignored (frozen)
let sealed = Object.seal({ x: 1 });
sealed.x = 2; sealed.y = 3;        // edit ok, add ignored
console.log(cfg.url, sealed);
```

Output:

```
Cat makes a sound | own property? false
api.com { x: 2 }
```

12. Classes, Inheritance, OOP, Frameworks

12.1 Person class with a method

Asked as: Lab S4.3 · Theory M2-Q9

```
class Person {
  constructor(name, age) {
    this.name = name;
    this.age = age;
  }
  introduce() {
    return "Hi, I am " + this.name + ", " + this.age + " years old.";
  }
}
let p = new Person("Amit", 21);
console.log(p.introduce());
```

Output:

```
Hi, I am Amit, 21 years old.
```

12.2 Extend to a Student class with marks

Asked as: Lab S4.4

```
class Person {
  constructor(name, age) { this.name = name; this.age = age; }
  introduce() { return "Hi, I am " + this.name; }
}
class Student extends Person {
  constructor(name, age, marks) {
    super(name, age);                    // call the parent constructor
    this.marks = marks;
  }
  average() {
    let sum = this.marks.reduce((s, m) => s + m, 0);
    return (sum / this.marks.length).toFixed(2);
  }
  introduce() {                          // method overriding
    return super.introduce() + ", my average is " + this.average();
  }
}
let s = new Student("Riya", 20, [85, 90, 78]);
console.log(s.introduce());
console.log(s instanceof Student, s instanceof Person);
```

Output:

```
Hi, I am Riya, my average is 84.33
true true
```

12.3 Object-oriented properties of JS: encapsulation, inheritance, polymorphism

Asked as: Lab S7.1

```
class Account {
  #balance = 0;                             // encapsulation: private field
  constructor(owner) { this.owner = owner; }
  deposit(amt) { if (amt > 0) this.#balance += amt; return this; }
  getBalance() { return this.#balance; }
}
class Animal {
  constructor(name) { this.name = name; }
  speak() { return this.name + " makes a sound"; }
}
class Dog extends Animal {                    // inheritance
  speak() { return this.name + " barks"; }    // polymorphism (overriding)
}
let acc = new Account("Amit").deposit(500).deposit(250);
console.log(acc.owner, acc.getBalance(), acc.balance);   // #balance is hidden
let pets = [new Animal("Cat"), new Dog("Tommy")];
for (let p of pets) console.log(p.speak());
console.log(new Dog("x") instanceof Animal);
```

Output:

```
Amit 750 undefined
Cat makes a sound
Tommy barks
true
```

12.4 UI refresh with a modern stack (React, no build tools)

Asked as: Lab S4.1

```
<!DOCTYPE html>
<html>
<body>
<div id="root"></div>
<script src="https://unpkg.com/react@18/umd/react.development.js"></script>
<script src="https://unpkg.com/react-dom@18/umd/react-dom.development.js"></script>
<script src="https://unpkg.com/@babel/standalone/babel.min.js"></script>
<script type="text/babel">
function Counter() {
  const [count, setCount] = React.useState(0);          // state
  return (
    <div>
      <h2>Count: {count}</h2>
      <button onClick={() => setCount(count + 1)}>+1</button>
      <button onClick={() => setCount(0)}>Reset</button>
    </div>
  );
}
ReactDOM.createRoot(document.getElementById("root")).render(<Counter />);
</script>
</body>
</html>
```

Output: clicking +1 updates the number straight away, with no page reload. React re-renders only the part that changed (the virtual DOM).

Vue version (equally short):

```
<div id="app"><h2>{{ count }}</h2><button @click="count++">+1</button></div>
<script src="https://unpkg.com/vue@3"></script>
<script>
  Vue.createApp({ data() { return { count: 0 }; } }).mount("#app");
</script>
```

12.5 Data persistence with localStorage

Asked as: Lab S4.2 · Theory M2-Q14

```
<!DOCTYPE html>
<html>
<body>
<input id="item" placeholder="Add item">
<button onclick="addItem()">Add</button>
<button onclick="clearAll()">Clear</button>
<ul id="list"></ul>
<script>
let items = JSON.parse(localStorage.getItem("items")) || [];   // load saved data
function render() {
  document.getElementById("list").innerHTML =
    items.map(i => "<li>" + i + "</li>").join("");
}
function addItem() {
  let v = document.getElementById("item").value.trim();
  if (v === "") return;
  items.push(v);
  localStorage.setItem("items", JSON.stringify(items));      // save (strings only)
  document.getElementById("item").value = "";
  render();
}
function clearAll() {
  items = [];
  localStorage.removeItem("items");
  render();
}
render();
</script>
</body>
</html>
```

Output: the items you add are still listed after you refresh or reopen the page, until you click Clear.

13. Built-in Objects: Boolean, Symbol, Error, Date, Map, Set, Cookies

13.1 Boolean object: truthy and falsy values

Asked as: Lab S9.3

```
let values = [0, 1, -1, "", "0", "false", " ", null, undefined, NaN, [], {}];
let labels = ["0", "1", "-1", '""', '"0"', '"false"', '" "', "null", "undefined", "NaN", "[]", "{}"];
for (let i = 0; i < values.length; i++) {
  console.log(labels[i].padEnd(10), "->", Boolean(values[i]));
}
console.log("new Boolean(false) is truthy:", Boolean(new Boolean(false)));
```

Output:

```
0          -> false
1          -> true
-1         -> true
""         -> false
"0"        -> true
"false"    -> true
" "        -> true
null       -> false
undefined  -> false
NaN        -> false
[]         -> true
{}         -> true
new Boolean(false) is truthy: true
```

13.2 Symbol as a hidden property

Asked as: Lab S9.4

```
function makeId() {
  return Symbol("id");                // every Symbol is unique
}
const id = makeId();
let user = { name: "Amit", age: 21 };
user[id] = 101;                       // hidden property
for (let key in user) console.log("for-in key:", key);   // id does not appear
console.log("Object.keys:", Object.keys(user));
console.log("Hidden value:", user[id]);
console.log("Unique?", makeId() === makeId());
```

Output:

```
for-in key: name
for-in key: age
Object.keys: ['name', 'age']
Hidden value: 101
Unique? false
```

13.3 Throw and catch a custom error, log the stack

Asked as: Lab S9.5 · Theory M5-Q7

```
class AgeError extends Error {
  constructor(message) {
    super(message);
    this.name = "AgeError";
  }
}
function checkAge(age) {
  if (age < 0) throw new AgeError("Age cannot be negative: " + age);
  return "Age OK: " + age;
}
try {
  console.log(checkAge(20));
  console.log(checkAge(-5));
} catch (e) {
  console.log(e.name + " -> " + e.message);
  console.log("Stack:", e.stack.split("\n")[0]);
  console.log("Is AgeError?", e instanceof AgeError);
} finally {
  console.log("finally always runs");
}
```

Output:

```
Age OK: 20
AgeError -> Age cannot be negative: -5
Stack: AgeError: Age cannot be negative: -5
Is AgeError? true
finally always runs
```

13.4 Built-in error types

Asked as: Theory M5-Q7

```
let tests = [
  () => undefinedVariable,          // ReferenceError
  () => null.name,                  // TypeError
  () => new Array(-1),              // RangeError
  () => JSON.parse("{bad json}"),   // SyntaxError
];
for (let t of tests) {
  try { t(); } catch (e) { console.log(e.name + ": " + e.message); }
}
```

Output:

```
ReferenceError: undefinedVariable is not defined
TypeError: Cannot read properties of null (reading 'name')
RangeError: Invalid array length
SyntaxError: Expected property name or '}' in JSON at position 1 (line 1 column 2)
```

13.5 Today's date as YYYY-MM-DD

Asked as: Lab S10.2 · Theory M5-Q5

```
let d = new Date(2026, 9, 28, 14, 5, 9);   // fixed date for the demo; use new Date() for today
let year = d.getFullYear();
let month = d.getMonth() + 1;              // months are 0-11
let day = d.getDate();
console.log("Year:", year, "Month:", month, "Day:", day);
let formatted = year + "-" + String(month).padStart(2, "0") + "-" + String(day).padStart(2, "0");
console.log("Formatted:", formatted);
console.log("Time:", String(d.getHours()).padStart(2, "0") + ":" +
                    String(d.getMinutes()).padStart(2, "0") + ":" +
                    String(d.getSeconds()).padStart(2, "0"));
console.log("Weekday (0=Sun):", d.getDay());
```

Output:

```
Year: 2026 Month: 10 Day: 28
Formatted: 2026-10-28
Time: 14:05:09
Weekday (0=Sun): 3
```

13.6 Map of student marks: add, delete, check, iterate

Asked as: Lab S10.3 · Theory M5-Q13

```
let marks = new Map();
marks.set("Amit", 85);                 // add
marks.set("Riya", 92);
marks.set("Kiran", 78);
console.log("Size:", marks.size);
console.log("Riya:", marks.get("Riya"));
console.log("Has Amit?", marks.has("Amit"));
marks.delete("Amit");                  // delete
console.log("Has Amit after delete?", marks.has("Amit"));
marks.forEach((value, key) => console.log(key + " => " + value));
```

Output:

```
Size: 3
Riya: 92
Has Amit? true
Has Amit after delete? false
Riya => 92
Kiran => 78
```

13.7 Set avoids duplicates: size and iteration

Asked as: Lab S10.5

```
let inputs = ["apple", "banana", "apple", "cherry", "banana"];   // e.g. from prompt()
let s = new Set();
for (let v of inputs) s.add(v);
s.add("apple");                        // duplicate ignored
console.log("Size:", s.size);
for (let v of s) console.log(v);
console.log("Has cherry?", s.has("cherry"));
console.log("Back to array:", [...s]);
```

Output:

```
Size: 3
apple
banana
cherry
Has cherry? true
Back to array: ['apple', 'banana', 'cherry']
```

13.8 JSON stringify / parse

Asked as: Theory M5-Q13

```
let obj = { name: "Amit", skills: ["JS", "CSS"], active: true };
let text = JSON.stringify(obj);
console.log(text, typeof text);
let back = JSON.parse(text);
console.log(back.skills[0], typeof back);
```

Output:

```
{"name":"Amit","skills":["JS","CSS"],"active":true} string
JS object
```

13.9 setTimeout and setInterval

Asked as: Theory M5-Q8

```
console.log("1: start");
setTimeout(() => console.log("3: after timeout (runs last)"), 0);
console.log("2: end of script");
// setInterval(fn, 1000) repeats every second; stop with clearInterval(id)
```

Output:

```
1: start
2: end of script
3: after timeout (runs last)
```

13.10 Set, retrieve and delete cookies

Asked as: Lab S9.6

```
<!DOCTYPE html>
<html>
<body>
<input id="cname" placeholder="name"> <input id="cvalue" placeholder="value">
<button onclick="setCookie()">Set</button>
<button onclick="getCookie()">Get</button>
<button onclick="deleteCookie()">Delete</button>
<p id="out"></p>
<script>
function setCookie() {
  let name = document.getElementById("cname").value;
  let value = document.getElementById("cvalue").value;
  let d = new Date();
  d.setTime(d.getTime() + 7 * 24 * 60 * 60 * 1000);          // expires in 7 days
  document.cookie = name + "=" + encodeURIComponent(value) +
                    "; expires=" + d.toUTCString() + "; path=/";
  show("Cookie set");
}
function getCookie() {
  let name = document.getElementById("cname").value;
  let parts = document.cookie.split("; ");                   // "a=1; b=2"
  for (let p of parts) {
    let [k, v] = p.split("=");
    if (k === name) return show(name + " = " + decodeURIComponent(v));
  }
  show("Not found. All cookies: " + document.cookie);
}
function deleteCookie() {
  let name = document.getElementById("cname").value;
  document.cookie = name + "=; expires=Thu, 01 Jan 1970 00:00:00 UTC; path=/";
  show("Cookie deleted");
}
function show(msg) { document.getElementById("out").innerHTML = msg; }
</script>
</body>
</html>
```

Output: Set saves the cookie; Get shows `user = Amit`; Delete removes it, so Get then shows *Not found*. Open the page through a local server (for example VS Code Live Server). Some browsers block cookies on `file://` pages.

14. BOM: window, navigator, location, history, screen

14.1 Browser info using navigator

Asked as: Lab S7.2

```
<script>
  document.write("Browser (userAgent): " + navigator.userAgent + "<br>");
  document.write("Language: " + navigator.language + "<br>");
  document.write("Platform: " + navigator.platform + "<br>");
  document.write("Online: " + navigator.onLine + "<br>");
  document.write("Cookies enabled: " + navigator.cookieEnabled);
</script>
```

Output (example):

```
Browser (userAgent): Mozilla/5.0 (Windows NT 10.0; Win64; x64) ... Chrome/130.0 ...
Language: en-US
Platform: Win32
Online: true
Cookies enabled: true
```

14.2 Change page with location.href, history.back()

Asked as: Lab S7.3

```
<button onclick="location.href = 'https://www.google.com'">Go to Google</button>
<button onclick="history.back()">Back</button>
<button onclick="history.forward()">Forward</button>
<button onclick="location.reload()">Reload</button>
<button onclick="location.replace('page2.html')">Replace (no back)</button>
<p id="info"></p>
<script>
  document.getElementById("info").innerHTML =
    "URL: " + location.href + "<br>Host: " + location.hostname +
    "<br>Path: " + location.pathname + "<br>History length: " + history.length;
</script>
```

Output: each button navigates as labelled, and the paragraph shows the current URL details.

14.3 Screen resolution using screen

Asked as: Lab S7.4

```
<script>
  document.write("Screen: " + screen.width + " x " + screen.height + "<br>");
  document.write("Available: " + screen.availWidth + " x " + screen.availHeight + "<br>");
  document.write("Colour depth: " + screen.colorDepth + " bits<br>");
  document.write("Browser window: " + window.innerWidth + " x " + window.innerHeight);
</script>
```

Output (example):

```
Screen: 1920 x 1080
Available: 1920 x 1040
Colour depth: 24 bits
Browser window: 1536 x 730
```

14.4 Window popups: alert, confirm, prompt

Asked as: Theory M2-Q5

```
<button onclick="demo()">Try popups</button>
<p id="out"></p>
<script>
function demo() {
  alert("This is an alert");                       // returns nothing
  let ok = confirm("Do you want to continue?");     // true / false
  let name = prompt("Enter your name", "Guest");    // string or null
  document.getElementById("out").innerHTML =
    "confirm: " + ok + " | prompt: " + name;
}
</script>
```

Output: three popups in turn, then for example `confirm: true | prompt: Amit`.

14.5 Explore the window object

Asked as: Lab S8.1 · Theory M3-Q25

```
<script>
  var siteName = "My Site";                       // global var becomes window.siteName
  document.write("window.siteName: " + window.siteName + "<br>");
  document.write("Inner size: " + window.innerWidth + " x " + window.innerHeight + "<br>");
  document.write("document === window.document: " + (document === window.document));
  // window.open("https://example.com", "_blank");   // open new window/tab
</script>
```

Output: `window.siteName: My Site`, the window size, and `document === window.document: true`.

15. DOM and Events

15.1 Build a DOM tree and use getElementById / querySelector

Asked as: Lab S8.2 · Theory M4-Q13 · Theory M4-Q14

```
<!DOCTYPE html>
<html>
<body>
  <div id="box">
    <h2 id="title">DOM Demo</h2>
    <p class="info">First paragraph</p>
    <p class="info">Second paragraph</p>
  </div>
  <pre id="out"></pre>
<script>
  let box = document.getElementById("box");                // by id
  let first = document.querySelector(".info");             // first match
  let all = document.querySelectorAll(".info");            // all matches (NodeList)
  let lines = [];
  lines.push("Title text: " + document.getElementById("title").textContent);
  lines.push("First .info: " + first.textContent);
  lines.push("Count of .info: " + all.length);
  lines.push("box children: " + box.children.length);
  lines.push("Parent of title: " + document.getElementById("title").parentNode.id);
  // walk the tree
  for (let child of box.children) lines.push("  <" + child.tagName.toLowerCase() + "> " + child.textContent);
  document.getElementById("out").textContent = lines.join("\n");
</script>
</body>
</html>
```

Output (shown on the page):

```
Title text: DOM Demo
First .info: First paragraph
Count of .info: 2
box children: 3
Parent of title: box
  <h2> DOM Demo
  <p> First paragraph
  <p> Second paragraph
```

15.2 Manipulate the DOM: create, add, change, remove

Asked as: Lab S8.1 · Theory M4-Q15 · Theory M4-Q18 · Theory M4-Q19

```
<ul id="list"><li>HTML</li><li>CSS</li></ul>
<button onclick="addItem()">Add</button>
<button onclick="removeLast()">Remove last</button>
<button onclick="restyle()">Style</button>
<p id="note">Original text</p>
<script>
function addItem() {
  let li = document.createElement("li");       // create
  li.textContent = "JavaScript";
  document.getElementById("list").appendChild(li);   // add
}
function removeLast() {
  let list = document.getElementById("list");
  if (list.lastElementChild) list.lastElementChild.remove();   // remove
}
function restyle() {
  let note = document.getElementById("note");
  note.style.color = "white";                  // change style
  note.style.backgroundColor = "teal";
  note.classList.toggle("big");                // add or remove a class
  note.setAttribute("title", "changed");       // change an attribute
  note.innerHTML = "<b>Bold</b> via innerHTML";   // parses HTML
  // note.textContent = "<b>x</b>"  would show the tags as plain text
}
</script>
```

Output: *Add* appends “JavaScript” to the list, *Remove last* deletes the last item, and *Style* turns the paragraph white on teal and makes the word Bold bold.

15.3 Add a row to a table dynamically

Asked as: Lab S8.3

```
<input id="name" placeholder="Name"> <input id="marks" type="number" placeholder="Marks">
<button onclick="addRow()">Add Row</button>
<table border="1" id="tbl">
  <tr><th>#</th><th>Name</th><th>Marks</th></tr>
</table>
<script>
function addRow() {
  let name = document.getElementById("name").value.trim();
  let marks = document.getElementById("marks").value;
  if (name === "" || marks === "") { alert("Fill both fields"); return; }
  let table = document.getElementById("tbl");
  let row = table.insertRow(-1);                 // -1 = at the end
  row.insertCell(0).textContent = table.rows.length - 1;
  row.insertCell(1).textContent = name;
  row.insertCell(2).textContent = marks;
  document.getElementById("name").value = "";
  document.getElementById("marks").value = "";
}
</script>
```

Output: each click adds a numbered row, for example `1 | Amit | 88`.

15.4 Change text on button click with addEventListener

Asked as: Lab S8.4

```
<h2 id="msg">Hello!</h2>
<button id="btn">Click me</button>
<script>
  let clicks = 0;
  document.getElementById("btn").addEventListener("click", function () {
    clicks++;
    document.getElementById("msg").textContent = "Text changed! Clicked " + clicks + " time(s)";
  });
</script>
```

Output: after the first click the heading reads *Text changed! Clicked 1 time(s)*, and the count goes up with each click.

15.5 Common events: mouse, keyboard, input, form submit

Asked as: Theory M5-Q11 · Theory M5-Q9

```
<div id="box" style="width:150px;height:60px;background:lightblue">Hover me</div>
<input id="typer" placeholder="Type here">
<form id="f"><input id="q"><button>Submit</button></form>
<p id="log"></p>
<script>
  let log = document.getElementById("log");
  let box = document.getElementById("box");
  box.addEventListener("mouseover", () => box.style.background = "orange");
  box.addEventListener("mouseout", () => box.style.background = "lightblue");
  document.getElementById("typer").addEventListener("input", e => log.textContent = "Typed: " +
e.target.value);
  document.getElementById("typer").addEventListener("keydown", e => { if (e.key === "Enter") log.textContent =
"Enter pressed"; });
  document.getElementById("f").addEventListener("submit", function (e) {
    e.preventDefault();                          // stop page reload
    log.textContent = "Submitted: " + document.getElementById("q").value;
  });
</script>
```

Output: the box turns orange on hover, the paragraph echoes what you type live, and submitting the form shows its value without reloading the page.

15.6 Event bubbling and event delegation

Asked as: Theory M4-Q17

```
<ul id="menu">
  <li>Home</li><li>About</li><li>Contact</li>
</ul>
<p id="out"></p>
<script>
  // one listener on the parent handles clicks on every <li> (even ones added later)
  document.getElementById("menu").addEventListener("click", function (e) {
    if (e.target.tagName === "LI") {
      document.getElementById("out").textContent = "You clicked " + e.target.textContent;
    }
  });
  // e.stopPropagation() inside a child's handler would stop the click reaching the parent
</script>
```

Output: clicking *About* shows *You clicked About*.

15.7 Stacked paragraphs that rise on hover (Section C Q2 a and b)

Asked as: Sec C-2a · Sec C-2b

```
<!DOCTYPE html>
<html>
<head>
<style>
  .para { position: absolute; width: 300px; height: 120px; padding: 10px;
          border: 2px solid black; }
  #p1 { top: 20px;  left: 20px;  background: lightblue;  z-index: 1; }
  #p2 { top: 60px;  left: 60px;  background: lightgreen; z-index: 2; }
  #p3 { top: 100px; left: 100px; background: pink;       z-index: 3; }
</style>
</head>
<body>
  <p class="para" id="p1" onmouseover="toTop(this)" onmouseout="toBack(this)">
    Paragraph 1: Lorem ipsum dolor sit amet, consectetur adipiscing elit.</p>
  <p class="para" id="p2" onmouseover="toTop(this)" onmouseout="toBack(this)">
    Paragraph 2: Sed do eiusmod tempor incididunt ut labore et dolore.</p>
  <p class="para" id="p3" onmouseover="toTop(this)" onmouseout="toBack(this)">
    Paragraph 3: Ut enim ad minim veniam, quis nostrud exercitation.</p>
<script>
  let original = {};                          // remembers each paragraph's z-index
  // (a) paragraph under the mouse rises to the top
  function toTop(el) {
    original[el.id] = getComputedStyle(el).zIndex;
    el.style.zIndex = 10;
  }
  // (b) on mouse out it returns to its ORIGINAL stacking position, not the bottom
  function toBack(el) {
    el.style.zIndex = original[el.id];
  }
</script>
</body>
</html>
```

Output: hovering over the visible part of paragraph 1 or 2 brings it fully to the front. When the mouse leaves, it drops back to its own original layer. (For part (a) only, setting `zIndex = 0` on mouse-out sends it to the bottom.)

15.8 Convert a NodeList / HTMLCollection to an array

Asked as: Theory M4-Q5

```
<p class="x">A</p><p class="x">B</p><p class="x">C</p>
<script>
  let nodes = document.querySelectorAll(".x");          // NodeList
  let arr = Array.from(nodes);                          // or [...nodes]
  document.write(arr.map(p => p.textContent).join(", "));
</script>
```

Output: `A, B, C`

16. Forms and Projects (Section B)

16.1 Form value + JS calculation: salary with commission

Asked as: Mid-sem style

```
<!DOCTYPE html>
<html>
<body>
<h3>Salary Calculator</h3>
<form>
  <label>Sales amount (Rs.): </label>
  <input type="number" id="sales">
  <button type="button" onclick="calc()">Calculate</button>
</form>
<p id="out"></p>
<script>
function calc() {
  let sales = Number(document.getElementById("sales").value);   // string -> number
  if (isNaN(sales) || sales < 0) {
    document.getElementById("out").innerHTML = "Enter a valid amount";
    return;
  }
  let fixed = 9000;
  let commission = sales * 0.10;                // 10% commission on sales
  let total = fixed + commission;
  document.getElementById("out").innerHTML =
    "Fixed: Rs. " + fixed + "<br>Commission: Rs. " + commission.toFixed(2) +
    "<br>Total salary: Rs. " + total.toFixed(2);
}
</script>
</body>
</html>
```

Output: sales of 15000 gives *Fixed: Rs. 9000, Commission: Rs. 1500.00, Total salary: Rs. 10500.00*. (If the question says commission only on the amount *above* 9000, use `commission = Math.max(0, sales - 9000) * 0.10`.)

16.2 Form + if-else: student result and grade

Asked as: Mid-sem style

```
<input type="number" id="m1" placeholder="Sub 1">
<input type="number" id="m2" placeholder="Sub 2">
<input type="number" id="m3" placeholder="Sub 3">
<button onclick="result()">Result</button>
<p id="out"></p>
<script>
function result() {
  let marks = ["m1", "m2", "m3"].map(id => Number(document.getElementById(id).value));
  if (marks.some(m => isNaN(m) || m < 0 || m > 100)) { alert("Marks must be 0-100"); return; }
  let total = marks.reduce((s, m) => s + m, 0);
  let pct = total / marks.length;
  let grade = pct >= 90 ? "A" : pct >= 75 ? "B" : pct >= 60 ? "C" : pct >= 40 ? "D" : "F";
  document.getElementById("out").innerHTML =
    "Total: " + total + " | Percentage: " + pct.toFixed(2) + "% | Grade: " + grade;
}
</script>
```

Output: 80, 90 and 70 give *Total: 240* | *Percentage: 80.00%* | *Grade: B*.

16.3 BMI calculator

Asked as: Sec B-3

```
<h3>BMI Calculator</h3>
Height (cm): <input type="number" id="h"><br>
Weight (kg): <input type="number" id="w"><br>
<button onclick="bmi()">Calculate</button>
<p id="out"></p>
<script>
function bmi() {
  let h = Number(document.getElementById("h").value) / 100;   // cm -> m
  let w = Number(document.getElementById("w").value);
  if (!(h > 0) || !(w > 0)) { document.getElementById("out").innerHTML = "Enter valid values"; return; }
  let b = w / (h * h);
  let category;
  if (b < 18.5) category = "Underweight";
  else if (b < 25) category = "Normal";
  else if (b < 30) category = "Overweight";
  else category = "Obese";
  document.getElementById("out").innerHTML = "BMI = " + b.toFixed(1) + " (" + category + ")";
}
</script>
```

Output: 175 cm and 70 kg give *BMI = 22.9 (Normal)*.

16.4 Digital clock

Asked as: Sec B-4

```
<h1 id="clock" style="font-family: monospace"></h1>
<script>
function showTime() {
  let d = new Date();
  let h = d.getHours(), m = d.getMinutes(), s = d.getSeconds();
  let ampm = h >= 12 ? "PM" : "AM";
  h = h % 12 || 12;                                  // 0 -> 12
  let pad = n => String(n).padStart(2, "0");
  document.getElementById("clock").textContent =
    pad(h) + ":" + pad(m) + ":" + pad(s) + " " + ampm;
}
showTime();
setInterval(showTime, 1000);                         // update every second
</script>
```

Output: a live clock such as `02:05:09 PM` that ticks every second.

16.5 Clock with alarm, stopwatch and timer

Asked as: Sec B-6

```
<h2 id="clock"></h2>
Alarm (HH:MM): <input type="time" id="alarm"> <button onclick="setAlarm()">Set</button>
<h3>Stopwatch: <span id="sw">0.0</span>s</h3>
<button onclick="swStart()">Start</button> <button onclick="swStop()">Stop</button>
<button onclick="swReset()">Reset</button>
<h3>Timer: <span id="tm">0</span>s</h3>
<input type="number" id="secs" placeholder="seconds"> <button onclick="startTimer()">Start timer</button>
<script>
let alarmTime = "", swValue = 0, swId = null, tmId = null;
setInterval(function () {
  let d = new Date();
  let hhmm = String(d.getHours()).padStart(2, "0") + ":" + String(d.getMinutes()).padStart(2, "0");
  document.getElementById("clock").textContent = d.toLocaleTimeString();
  if (alarmTime === hhmm && d.getSeconds() === 0) alert("Alarm! " + alarmTime);
}, 1000);
function setAlarm() { alarmTime = document.getElementById("alarm").value; alert("Alarm set for " + alarmTime);
}
function swStart() { if (!swId) swId = setInterval(() => { swValue += 0.1;
document.getElementById("sw").textContent = swValue.toFixed(1); }, 100); }
function swStop() { clearInterval(swId); swId = null; }
function swReset() { swStop(); swValue = 0; document.getElementById("sw").textContent = "0.0"; }
function startTimer() {
  let left = Number(document.getElementById("secs").value);
  clearInterval(tmId);
  document.getElementById("tm").textContent = left;
  tmId = setInterval(function () {
    left--;
    document.getElementById("tm").textContent = left;
    if (left <= 0) { clearInterval(tmId); alert("Time up!"); }
  }, 1000);
}
</script>
```

Output: a live clock, an alarm popup at the set minute, a stopwatch counting in tenths of a second, and a countdown timer that ends with *Time up!*

16.6 Calculator

Asked as: Sec B-8

```
<style>
  #calc { width: 220px; } #disp { width: 100%; height: 40px; font-size: 20px; text-align: right; }
  #calc button { width: 52px; height: 45px; font-size: 18px; margin: 1px; }
</style>
<div id="calc">
  <input id="disp" readonly>
  <div id="keys"></div>
</div>
<script>
let keys = ["7","8","9","/","4","5","6","*","1","2","3","-","0",".","=","+","C"];
let disp = document.getElementById("disp");
for (let k of keys) {
  let b = document.createElement("button");
  b.textContent = k;
  b.onclick = () => press(k);
  document.getElementById("keys").appendChild(b);
}
function press(k) {
  if (k === "C") disp.value = "";
  else if (k === "=") disp.value = evaluate(disp.value);
  else disp.value += k;
}
function evaluate(expr) {                 // left-to-right, no eval()
  let tokens = expr.match(/(\d+\.?\d*|[+\-*/])/g);
  if (!tokens) return "";
  let result = Number(tokens[0]);
  for (let i = 1; i < tokens.length; i += 2) {
    let op = tokens[i], n = Number(tokens[i + 1]);
    if (op === "+") result += n;
    else if (op === "-") result -= n;
    else if (op === "*") result *= n;
    else if (op === "/") result = n === 0 ? NaN : result / n;
  }
  return isNaN(result) ? "Error" : result;
}
</script>
```

Output: button grid; `12 + 3 * 2 =` gives `30`. This version calculates left to right; say so in the exam, or use `eval(disp.value)` if operator precedence is required.

16.7 Notes / To-Do list (add, edit, delete, mark done, save)

Asked as: Sec B-9 · Sec B-17

```
<h3>To-Do List</h3>
<input id="task" placeholder="New task"> <button onclick="addTask()">Add</button>
<ul id="list"></ul>
<style> .done { text-decoration: line-through; color: gray; } li button { margin-left: 6px; } </style>
<script>
let tasks = JSON.parse(localStorage.getItem("tasks")) || [];
function save() { localStorage.setItem("tasks", JSON.stringify(tasks)); }
function render() {
  let ul = document.getElementById("list");
  ul.innerHTML = "";
  tasks.forEach(function (t, i) {
    let li = document.createElement("li");
    let span = document.createElement("span");
    span.textContent = t.text;
    if (t.done) span.className = "done";
    span.onclick = () => { t.done = !t.done; save(); render(); };          // mark done
    let edit = document.createElement("button");
    edit.textContent = "Edit";
    edit.onclick = () => { let v = prompt("Edit task", t.text); if (v) { t.text = v; save(); render(); } };
    let del = document.createElement("button");
    del.textContent = "Delete";
    del.onclick = () => { tasks.splice(i, 1); save(); render(); };
    li.append(span, edit, del);
    ul.appendChild(li);
  });
}
function addTask() {
  let input = document.getElementById("task");
  if (input.value.trim() === "") return;
  tasks.push({ text: input.value.trim(), done: false });
  input.value = "";
  save(); render();
}
render();
</script>
```

Output: tasks appear with Edit and Delete buttons. Clicking a task strikes it through, and the list survives a page refresh because it’s stored in localStorage.

16.8 Quiz app with score and correct answers

Asked as: Sec B-12

```
<div id="quiz"></div>
<button onclick="submitQuiz()">Submit</button>
<div id="result"></div>
<script>
let questions = [
  { q: "Which tag links an external JS file?", options: ["<js>", "<script>", "<link>"], answer: 1 },
  { q: "typeof null is?", options: ["null", "undefined", "object"], answer: 2 },
  { q: "Which method adds to the end of an array?", options: ["push", "pop", "shift"], answer: 0 }
];
let html = "";
questions.forEach(function (item, i) {
  html += "<p><b>" + (i + 1) + ". " + item.q.replace(/</g, "&lt;") + "</b><br>";
  item.options.forEach(function (opt, j) {
    html += '<label><input type="radio" name="q' + i + '" value="' + j + '"> ' +
            opt.replace(/</g, "&lt;") + "</label><br>";
  });
  html += "</p>";
});
document.getElementById("quiz").innerHTML = html;
function submitQuiz() {
  let score = 0, report = "";
  questions.forEach(function (item, i) {
    let chosen = document.querySelector('input[name="q' + i + '"]:checked');
    let picked = chosen ? Number(chosen.value) : -1;
    if (picked === item.answer) score++;
    else report += "Q" + (i + 1) + ": correct answer is " + item.options[item.answer].replace(/</g, "&lt;") +
"<br>";
  });
  document.getElementById("result").innerHTML =
    "<h3>Score: " + score + "/" + questions.length + "</h3>" + report;
}
</script>
```

Output: radio-button questions; Submit shows something like *Score: 2/3* plus the correct answer for each one you got wrong. (Add a countdown with `setInterval` as in 16.5 for a timed quiz.)

16.9 Random quote generator

Asked as: Sec B-16

```
<blockquote id="quote" style="font-size:20px"></blockquote>
<button onclick="newQuote()">New Quote</button>
<script>
let quotes = [
  { text: "Simplicity is the soul of efficiency.", by: "Austin Freeman" },
  { text: "First, solve the problem. Then, write the code.", by: "John Johnson" },
  { text: "Code is like humor. When you have to explain it, it's bad.", by: "Cory House" }
];
let lastIndex = -1;
function newQuote() {
  let i;
  do { i = Math.floor(Math.random() * quotes.length); } while (i === lastIndex);  // no repeat
  lastIndex = i;
  document.getElementById("quote").innerHTML = '"' + quotes[i].text + '"<br>- ' + quotes[i].by;
}
newQuote();
</script>
```

Output: a random quote with its author; each click shows a different one.

16.10 Scramble word game

Asked as: Sec B-7

```
<h3>Unscramble: <span id="jumbled"></span></h3>
<input id="guess"> <button onclick="check()">Check</button> <button onclick="next()">New word</button>
<p id="msg"></p>
<script>
let words = ["javascript", "browser", "function", "variable", "element"];
let current = "";
function shuffle(word) {
  let a = word.split("");
  for (let i = a.length - 1; i > 0; i--) {              // Fisher-Yates
    let j = Math.floor(Math.random() * (i + 1));
    [a[i], a[j]] = [a[j], a[i]];
  }
  let s = a.join("");
  return s === word ? shuffle(word) : s;                 // never show the answer
}
function next() {
  current = words[Math.floor(Math.random() * words.length)];
  document.getElementById("jumbled").textContent = shuffle(current);
  document.getElementById("guess").value = "";
  document.getElementById("msg").textContent = "";
}
function check() {
  let g = document.getElementById("guess").value.trim().toLowerCase();
  document.getElementById("msg").textContent =
    g === current ? "Correct! You pass the level." : "Sorry, Please try again";
}
next();
</script>
```

Output: shows a jumble such as `pvtscjiaar`. Typing `javascript` gives *Correct! You pass the level.*; anything else gives *Sorry, Please try again*.

16.11 Budget application

Asked as: Sec B-10

```
<h3>Monthly Budget</h3>
Month: <input type="month" id="month">
Budget: <input type="number" id="budget"><br>
Expense: <input id="desc" placeholder="description"> <input type="number" id="amt" placeholder="amount">
<button onclick="addExpense()">Add</button>
<ul id="list"></ul>
<p id="summary"></p>
<script>
let data = JSON.parse(localStorage.getItem("budget")) || {};      // { "2026-10": {budget, expenses: []} }
function key() { return document.getElementById("month").value || "default"; }
function current() { return data[key()] || (data[key()] = { budget: 0, expenses: [] }); }
function addExpense() {
  let m = current();
  m.budget = Number(document.getElementById("budget").value) || m.budget;
  let desc = document.getElementById("desc").value.trim();
  let amt = Number(document.getElementById("amt").value);
  if (desc && amt > 0) m.expenses.push({ desc, amt });
  localStorage.setItem("budget", JSON.stringify(data));
  show();
}
function show() {
  let m = current();
  document.getElementById("list").innerHTML =
    m.expenses.map(e => "<li>" + e.desc + ": Rs. " + e.amt + "</li>").join("");
  let spent = m.expenses.reduce((s, e) => s + e.amt, 0);
  document.getElementById("summary").innerHTML =
    "Budget: Rs. " + m.budget + " | Spent: Rs. " + spent + " | Left: Rs. " + (m.budget - spent);
}
document.getElementById("month").onchange = show;
</script>
```

Output: with a budget of 10000 and expenses Rent 6000 and Food 2500, it shows *Budget: Rs. 10000 | Spent: Rs. 8500 | Left: Rs. 1500*. Each month is saved separately in localStorage. A login screen could wrap this, as in the to-do app.

16.12 Text-to-speech and speech detection (Web Speech API)

Asked as: Sec B-2

```
<textarea id="text" rows="3" cols="40">Hello, welcome to the front end lab.</textarea><br>
<button onclick="speak()">Speak</button>
<button onclick="listen()">Start listening</button>
<p id="heard"></p>
<script>
function speak() {
  let msg = new SpeechSynthesisUtterance(document.getElementById("text").value);
  msg.rate = 1; msg.pitch = 1;
  window.speechSynthesis.speak(msg);
}
function listen() {
  let SR = window.SpeechRecognition || window.webkitSpeechRecognition;
  if (!SR) { alert("Speech recognition not supported (use Chrome)"); return; }
  let rec = new SR();
  rec.lang = "en-IN";
  rec.onresult = e => document.getElementById("heard").textContent = "You said: " + e.results[0]
[0].transcript;
  rec.start();
}
</script>
```

Output: Speak reads the text aloud. Start listening asks for microphone access, then shows *You said: …* (Chrome).

16.13 Single Page App (hash routing, no reload)

Asked as: Sec B-5

```
<nav><a href="#home">Home</a> | <a href="#about">About</a> | <a href="#contact">Contact</a></nav>
<div id="view"></div>
<script>
let pages = {
  home: "<h2>Home</h2><p>Welcome to the SPA.</p>",
  about: "<h2>About</h2><p>All pages load inside one HTML file.</p>",
  contact: "<h2>Contact</h2><p>Email: hello@example.com</p>"
};
function route() {
  let page = location.hash.slice(1) || "home";
  document.getElementById("view").innerHTML = pages[page] || "<h2>404</h2>";
}
window.addEventListener("hashchange", route);      // runs when the link changes the #hash
route();
</script>
```

Output: clicking the links swaps the content instantly without reloading the page, and the browser’s Back button still works.

16.14 Mini e-commerce: products, search, cart, order

Asked as: Sec B-11

```
<input id="search" placeholder="Search products" oninput="showProducts()">
<div id="products"></div>
<h3>Cart</h3><ul id="cart"></ul><p id="total"></p>
<button onclick="placeOrder()">Order / Pay</button>
<script>
let products = [
  { id: 1, name: "Laptop", price: 50000 }, { id: 2, name: "Mouse", price: 500 },
  { id: 3, name: "Keyboard", price: 1500 }, { id: 4, name: "Monitor", price: 9000 }
];
let cart = [];
function showProducts() {
  let q = document.getElementById("search").value.toLowerCase();
  document.getElementById("products").innerHTML = products
    .filter(p => p.name.toLowerCase().includes(q))
    .map(p => p.name + " - Rs. " + p.price + ' <button onclick="addToCart(' + p.id + ')">Add</button>')
    .join("<br>");
}
function addToCart(id) {
  cart.push(products.find(p => p.id === id));
  showCart();
}
function showCart() {
  document.getElementById("cart").innerHTML = cart.map(p => "<li>" + p.name + "</li>").join("");
  let total = cart.reduce((s, p) => s + p.price, 0);
  document.getElementById("total").textContent = "Total: Rs. " + total;
}
function placeOrder() {
  if (cart.length === 0) return alert("Cart is empty");
  if (confirm("Pay Rs. " + cart.reduce((s, p) => s + p.price, 0) + "?")) {   // payment placeholder
    alert("Order placed! Items: " + cart.length);
    cart = []; showCart();
  }
}
showProducts();
</script>
```

Output: a product list filtered live by the search box; Add puts items in the cart with a running total; Order asks for confirmation and then clears the cart.

16.15 Movie app: search, description, wishlist

Asked as: Sec B-13

```
<input id="q" placeholder="Search movie"> <button onclick="search()">Search</button>
<div id="results"></div><h3>Wishlist</h3><ul id="wish"></ul>
<script>
// Offline sample data. With a real API (e.g. OMDb, free key) replace getMovies with:
// fetch("https://www.omdbapi.com/?apikey=KEY&s=" + q).then(r => r.json()).then(d => d.Search)
let movies = [
  { title: "Inception", year: 2010, plot: "A thief steals secrets through dreams." },
  { title: "Interstellar", year: 2014, plot: "Explorers travel through a wormhole." },
  { title: "3 Idiots", year: 2009, plot: "Engineering friends challenge the system." }
];
let wishlist = JSON.parse(localStorage.getItem("wish")) || [];
function search() {
  let q = document.getElementById("q").value.toLowerCase();
  let found = movies.filter(m => m.title.toLowerCase().includes(q));
  document.getElementById("results").innerHTML = found.map(m =>
    "<p><b>" + m.title + "</b> (" + m.year + ")<br>" + m.plot +
    ' <button onclick="addWish(\'' + m.title + '\')">+ Wishlist</button></p>').join("") || "No results";
}
function addWish(title) {
  if (!wishlist.includes(title)) wishlist.push(title);
  localStorage.setItem("wish", JSON.stringify(wishlist));
  showWish();
}
function showWish() {
  document.getElementById("wish").innerHTML = wishlist.map(t => "<li>" + t + "</li>").join("");
}
showWish();
</script>
```

Output: searching “inter” shows Interstellar with its plot; *+ Wishlist* adds it to a saved wishlist.

16.16 File sharing with encryption (Web Crypto AES)

Asked as: Sec B-14

```
<input type="file" id="file"> <input id="pass" type="password" placeholder="Password">
<button onclick="encrypt()">Encrypt & download</button>
<button onclick="decrypt()">Decrypt & download</button>
<p id="msg"></p>
<script>
async function getKey(password, salt) {
  let base = await crypto.subtle.importKey("raw", new TextEncoder().encode(password), "PBKDF2", false,
["deriveKey"]);
  return crypto.subtle.deriveKey({ name: "PBKDF2", salt, iterations: 100000, hash: "SHA-256" },
    base, { name: "AES-GCM", length: 256 }, false, ["encrypt", "decrypt"]);
}
function download(bytes, name) {
  let a = document.createElement("a");
  a.href = URL.createObjectURL(new Blob([bytes]));
  a.download = name; a.click();
}
async function encrypt() {
  let f = document.getElementById("file").files[0];
  let salt = crypto.getRandomValues(new Uint8Array(16)), iv = crypto.getRandomValues(new Uint8Array(12));
  let key = await getKey(document.getElementById("pass").value, salt);
  let data = await crypto.subtle.encrypt({ name: "AES-GCM", iv }, key, await f.arrayBuffer());
  download(new Blob([salt, iv, data]), f.name + ".enc");            // salt + iv + ciphertext
  document.getElementById("msg").textContent = "Encrypted: " + f.name + ".enc";
}
async function decrypt() {
  let f = document.getElementById("file").files[0];
  let all = new Uint8Array(await f.arrayBuffer());
  let key = await getKey(document.getElementById("pass").value, all.slice(0, 16));
  try {
    let data = await crypto.subtle.decrypt({ name: "AES-GCM", iv: all.slice(16, 28) }, key, all.slice(28));
    download(data, f.name.replace(/\.enc$/, ""));
    document.getElementById("msg").textContent = "Decrypted successfully";
  } catch (e) { document.getElementById("msg").textContent = "Wrong password or corrupted file"; }
}
</script>
```

Output: encrypting downloads `photo.jpg.enc`, which can be shared safely. Decrypting it with the same password gives back the original file; a wrong password shows an error. (Sending files between users needs a server or storage, which is outside a front-end lab.)

16.17 Real-time chat (Node.js + Socket.io)

Asked as: Sec B-15

```
// server.js   ->  npm install express socket.io   then   node server.js
const app = require("express")();
const http = require("http").createServer(app);
const io = require("socket.io")(http);
app.get("/", (req, res) => res.sendFile(__dirname + "/index.html"));
io.on("connection", socket => {
  socket.on("chat", msg => io.emit("chat", msg));     // broadcast to everyone
});
http.listen(3000, () => console.log("Open http://localhost:3000"));
```

```
<!-- index.html -->
<ul id="msgs"></ul>
<input id="name" placeholder="Name"> <input id="m" placeholder="Message">
<button onclick="send()">Send</button>
<script src="/socket.io/socket.io.js"></script>
<script>
  const socket = io();
  function send() {
    let text = document.getElementById("m").value;
    if (text) socket.emit("chat", document.getElementById("name").value + ": " + text);
    document.getElementById("m").value = "";
  }
  socket.on("chat", function (msg) {
    let li = document.createElement("li");
    li.textContent = msg;
    document.getElementById("msgs").appendChild(li);
  });
</script>
```

Output: open the page in two browser tabs. A message sent from one appears instantly in both.

16.18 Form validation page

Asked as: Sec B-1

The complete form validation page (name, email, password, confirm password, mobile) is **10.6**, and the USN validator is **10.7**.